Nome do Projeto: P.G.S (Projeto do Grupo Seis)

Nome do Grupo: O.G.S (O Grupo Seis)

Integrantes:

- Arthur Bretas Oliveira
- Davi D'avila Versan
- Gabriel Santos do Nascimento
- Nicolas Ramon da Silva
- Otavio de Carvalho Vasconcelos
- Vitor Margarido Balbo
- Vinicius dos Reis Savian

Sumário

- Contextualização de BD/DN, análises de qualidade e decisões arquiteturais
  - Business drivers extraídos do TAP (o que o parceiro espera)
  - O que significa cada análise (como BD/DN viram evidência e arquitetura)
  - Priorização e transparência de trade-offs (aspectos “+” e “–” além do foco)
- 1. Business Drivers
  - 1.1 DN1 (Blue Team) - Latência de Ingestão
    - Especificação da API_MOCK_G6
    - Métricas de SLA e Alertas
    - Metodologia de Testagem (Blue Team)
    - Resultados Obtidos
    - Decisão Arquitetural e Contexto de BD/DN
    - Impacto na Qualidade — Aspectos Positivos e Negativos Mapeados
  - 1.2 DN2 (Blue Team) - Integridade dos Dados Fiscais na Ingestão
    - Objetivo
    - Regras de Negócio
    - Especificação Técnica
    - Metodologia de Testagem (TDD)
    - Resultados Obtidos
    - Execução dos Testes
    - Decisão Arquitetural e Contexto de BD/DN
    - Impacto na Qualidade — Aspectos Positivos e Negativos Mapeados
  - 1.3 DN3 (Blue Team) - Disponibilidade e Resiliência da API
    - Objetivo
    - Regras de Negócio
    - Especificação da API — Endpoints de Resiliência
    - Mapa de Direcionadores de Negócio — Blue Team (DN3)
    - Implementação Técnica
    - Metodologia de Testagem (Blue Team — TDD)
    - Resultados Obtidos
    - Execução dos Testes
    - Decisão Arquitetural e Contexto de BD/DN
    - Impacto na Qualidade — Aspectos Positivos e Negativos Mapeados

    - 1.4 DN4 (Blue Team) - Não-Repúdio de Transações de Auditoria Fiscal
        - Objetivo
        - Especificação da API — Endpoint de Não-Repúdio
        - Mapa de Direcionadores de Negócio — Blue Team (DN4)
        - Implementação Técnica
        - Metodologia de Testagem (Blue Team — TDD)
        - Resultados Obtidos
        - Execução dos Testes
        - Decisão Arquitetural e Contexto de BD/DN
        - Impacto na Qualidade — Aspectos Positivos e Negativos Mapeados
- 2. Direcionadores Adversariais (Red Team)
    - 2.1 DN1 (Red Team) - Testes Adversariais de Latência (SLA)
        - Objetivo do Ativo Adversarial
        - Critérios de Ataque
        - Bateria de Testes e Evolução da Carga
        - Métricas e Logs
        - Resultados Obtidos
        - Síntese de Impacto (Positivos x Negativos) e Priorizações
        - Execução
        - Execução dos Testes (API real) — DN1
    - 2.2 DN2 (Red Team) - Integridade dos Dados Fiscais na Ingestão
        - Objetivo do Ativo Adversarial
        - Mapa de Direcionadores de Negócio - Red Team (DN2)
        - Métodos de Testagem
        - Execução dos Testes (API real)
        - Análise dos Resultados (Logs de Execução)
        - Síntese de Impacto (Positivos x Negativos) e Priorizações
        - Argumentação e Ações Corretivas
    - 2.3 DN4 (Red Team) - Não-Repúdio sob Perspectiva Adversarial
        - Objetivo do Ativo Adversarial
        - Mapa de Direcionadores de Negócio - Red Team (DN4)
        - Métodos de Testagem
        - Execução dos Testes (API real) — DN4
        - Análise de Resultados
        - Síntese de Impacto (Positivos x Negativos) e Priorizações
        - Ações Corretivas Recomendadas
    - 2.4 DN3 (Red Team) - Disponibilidade e Resiliência da API
        - Objetivo do Ativo Adversarial
        - Mapa de Direcionadores de Negócio - Red Team (DN3)
        - Métodos de Testagem e Execução
        - Execução dos Testes (API real) — DN3
        - Análise dos Resultados — Pontos Fortes
        - Pontos de Quebra Identificados (FAIL)
        - Síntese de Impacto (Positivos x Negativos) e Priorizações
- 3. Requisitos de Software (ISO 25010)
    - 3.1 Requisitos Funcionais
        - RF1 — Consulta de Status de Processamento Assíncrono
        - RF2 — Validação de Módulos Fiscais de Destino
    - 3.2 Requisitos Não Funcionais
        - RNF1 — Eficiência de Desempenho: Capacidade (ISO 25010)
        - RNF2 — Segurança: Autenticidade (ISO 25010)
        - RNF3 — Manutenibilidade: Testabilidade (ISO 25010)
    - 3.3 RNF — Verificação de Não-Repúdio a partir de Reclamação de Cliente
        - Definição do Requisito
        - Especificação da API — Endpoint de Disputa
        - Mapa de Direcionadores — RNF1
        - Implementação Técnica — Motor de Sazonalidade
        - Metodologia de Testagem (Blue Team — TDD)
        - Resultados Obtidos
        - Execução dos Testes
        - Decisão Arquitetural e Contexto de BD/DN
        - Impacto na Qualidade — Aspectos Positivos e Negativos Mapeados
- 4. Integrações de Serviços (Sprint de Consolidação)
    - 4.1 INT1 — Análise Estática com SonarCloud
    - 4.2 - INT2: Locust

        - Objetivo
        - Cenários de Teste (CEN-1 a CEN-5)
        - Método de Teste
        - Métricas Obtidas
        - Relação com Business Drivers e Requisitos
        - Guia de Execução e Interpretação de Resultados
        - Resultados Comparativos: 100 vs 1.000 Usuários Simultâneos
    - 4.3 INT3 (Blue Team) — Integração de Gestão de Identidade com Clerk
        - Objetivo
        - Regras de Negócio
        - Especificação da API — Endpoint de Autenticação Híbrida
        - Mapa de Direcionadores de Negócio — Blue Team (INT3)
        - Implementação Técnica
        - Metodologia de Testagem (Blue Team — TDD)
        - Resultados Obtidos
        - Execução dos Testes
        - Decisão Arquitetural e Contexto de BD/DN
        - Configuração de Variáveis de Ambiente
        - Impacto na Qualidade — Aspectos Positivos e Negativos Mapeados
    - 4.4 INT4 — Sincronismo de versões via API do GitLab
        - 4.4.1 Assinatura / critério de versão
        - 4.4.2 Objetivo da integração
        - 4.4.3 Requisitos e DNs relacionados
        - 4.4.4 Como se conectar ao serviço
        - 4.4.5 Função de verificação do protocolo
        - 4.4.6 Testes
    - 4.5 INT5 (Blue Team) — Aferição de Layout SPED/RFB
- 5. Direcionadores Adversariais — Red Team (Sprint de Consolidação)
    - 5.1 INT3 (Red Team) — Autenticação Híbrida Clerk sob Perspectiva Adversarial
    - 5.2 RNF1 (Red Team) — Disputa de Não-Repúdio sob Perspectiva Adversarial
    - 5.3 RNF2 (Red Team) — Autenticidade sob Perspectiva Adversarial
    - 5.4 INT4 (Red Team) — Sincronismo GitLab sob Perspectiva Adversarial
    - 5.5 INT5 (Red Team) — Aferição SPED/RFB sob Perspectiva Adversarial
- 6. Dashboard e Observabilidade
    - 6.1 Prometheus
    - 6.2 Grafana e Dashboards Executivos
- 7. Consolidação Final da API e do Controle de Qualidade
    - 7.1 Ajustes finais de implementação da API
    - 7.2 Análise do repositório: programas de controle de qualidade

Contextualização de BD/DN, análises de qualidade e decisões arquiteturais

Este projeto segue o direcionamento do TAP, cujo problema central, em um cenário **API First**, é garantir **cobertura de testes automatizada com qualidade e relevância** — isto é, não apenas "testar mais", mas validar comportamento correto sob diferentes condições e reduzir risco em mudanças e deploys.

Para atender esse objetivo, o documento organiza o trabalho em **Business Drivers (BD)** e **Direcionadores de Negócio (DN)**. Aqui, BD/DN são usados como uma ponte entre **risco/valor de negócio** e **evidência técnica** (testes, métricas, relatórios e decisões de arquitetura). Em outras palavras, cada análise descrita nas seções seguintes responde a duas perguntas: *qual risco do parceiro estamos reduzindo?* e *qual evidência entregamos para sustentar essa redução?*

Business drivers extraídos do TAP (o que o parceiro espera)

Do TAP, derivam-se os principais drivers que orientam o recorte e a forma de avaliação:

- **Confiabilidade e estabilidade dos serviços**: capacidade de manter o fluxo operacional correto, mesmo sob variação de carga, entradas inválidas e condições adversas.
- **Velocidade com confiança no deploy**: testes automatizados que permitem evolução de funcionalidades sem regressão, com feedback rápido.

- **Qualidade e relevância da cobertura**: foco em cenários críticos e "comportamento da API", não apenas porcentagem de linhas cobertas.
- **Observabilidade e relatórios acionáveis**: resultados com detalhe suficiente para tomada de decisão (o que falhou, por quê, impacto e prioridade).
- **Evolução para esteira de CI/CD e replicabilidade**: execução local/CLI e estrutura que possa ser reaplicada a outros módulos, como previsto no escopo macro/MVP.
- **Análise de qualidade, arquitetura e segurança**: insumos técnicos que apoiem decisões e governança, sem expor conteúdo restrito do parceiro.

Para garantir rastreabilidade entre expectativa e execução, as análises BD/DN apresentadas no documento são organizadas para cobrir esses drivers com evidências verificáveis:

Driver (TAP)	Como aparece nas análises (BD/DN)	Evidência gerada no documento
Confiabilidade e estabilidade	Integridade de payload, disponibilidade/resiliência e não-repúdio	Casos de teste, critérios de aceite, resultados (pass/fail), limitações e recomendações
Velocidade com confiança no deploy	Automação de testes orientada a dados (TDD) e execução repetível	Procedimentos de execução, suíte automatizada e resultados reprodutíveis
Relevância da cobertura	Priorização por risco de negócio e contrato da API	Justificativa de escopo, mapeamento de "+" e "–", riscos fora do foco e próximos passos
Observabilidade e relatórios acionáveis	Métricas, SLAs, alertas e health checks	Métricas reportadas (ex.: conformidade), alertas/limiares e interpretação de impacto
CI/CD e replicabilidade	Estrutura e disciplina de execução local/CLI visando pipeline	Organização do trabalho e instruções de execução compatíveis com esteira
Qualidade/arquitetura/segurança	Avaliação de trade-offs arquiteturais sob risco	Decisões arquiteturais vinculadas a riscos e evidências de mitigação

O que significa cada análise (como BD/DN viram evidência e arquitetura)

As análises de BD/DN neste documento são estruturadas para explicitar pontos positivos (fluxos, intensidade, precisão, base para automação/ML) e também limitações/riscos. Abaixo, o significado de cada eixo de análise e sua implicação arquitetural:

- **Latência / desempenho (ex.: DN de ingestão)**: mede se o fluxo de auditoria mantém tempo de resposta compatível com operação real. Implica decisões como desenho de endpoints, controle de processamento assíncrono, limiares de alerta e critérios de aceite.
- **Integridade / correção de dados**: valida se o sistema rejeita entradas inválidas antes de consumir recursos e antes de gerar efeitos colaterais (ex.: rastreio/processamento). Implica validação de contrato, regras de negócio explícitas e testes orientados a dados (TDD).
- **Disponibilidade / resiliência**: avalia se a API se mantém operante sob falhas e abuso, com degradação controlada. Implica mecanismos como rate limiting, circuit breaker, health check, idempotência e alertas.
- **Segurança e auditabilidade (ex.: não-repúdio)**: mede evidências de autoria/imutabilidade e a robustez contra replays e adulterações. Implica decisões criptográficas, encadeamento de logs, controles de acesso e critérios de auditoria.

Priorização e transparência de trade-offs (aspectos "+" e "–" além do foco)

Para evitar uma visão “estreita” do que foi testado, cada seção busca mapear **aspectos positivos e negativos**, inclusive itens fora do foco imediato, e justificar a priorização aplicada. Isso tem dois objetivos: (1) deixar claro ao parceiro o que já reduz risco de forma comprovada e (2) registrar o que ficou como recomendação por limitações de tempo, conhecimento, acesso à plataforma e infraestrutura.

Assim, o conjunto BD/DN não serve apenas para descrever testes; ele explicita **decisões arquiteturais advindas de riscos de negócio**, os ganhos esperados (fluxo, intensidade e precisão) e a racionalidade de priorização do projeto.

1. Business Drivers

1.1 DN1 (Blue Team) - Latência de Ingestão

- SLA: 90% dos arquivos de auditoria devem gerar um ID de processo em até 500ms (P90 <= 500ms).
- Este direcionador simula o processo em que o sistema Kolossus receba e processe lotes de documentos fiscais dentro dos parâmetros de performance necessários para não impactar a operação dos clientes.

Especificação da API_MOCK_G6

A API foi projetada para seguir a estrutura de referência exigida, mapeando as necessidades da ASIS:

- sessao_cliente: Identificador único da sessão de auditoria.
- valor: Volume de documentos fiscais submetidos no lote.
- destino: Módulo fiscal de destino (ex: 'EFD_ICMS_IPI', 'NCM_AUDIT').

Métricas de SLA e Alertas

- SLA de Latência: ≤ 500 ms (90% das requisições).
- Alerta de Lentidão Crítica: Disparo automático de sinalização caso o tempo exceda 650 ms (30% acima do SLA).
- Status de Efetivação: A transação é considerada válida apenas com o retorno HTTP 201 Created.

Metodologia de Testagem (Blue Team)

A aferição foi conduzida utilizando uma abordagem Test Data Driven (TDD) através do script src/tests/blue_team/response_time.py.

Serviços de Apoio: Implementação de mecanismos de Autenticação (account-key/app-key), Rastreio (geração de ID de processo) e Efetivação (validação de persistência assíncrona).

Ambiente: Execução via CLI conectada a um servidor Mock que replica o comportamento assíncrono da ASIS API.

Resultados Obtidos

A suíte de testes validou a resiliência do sistema com os seguintes indicadores:

- Status Final: APROVADO
- Taxa de Conformidade: 100.00%

- Latência Média: 373.43 ms
- Comportamento de Alerta: O código monitorou proativamente os limites, não detectando violações de 650 ms durante o ciclo de teste.

Decisão Arquitetural e Contexto de BD/DN

A escolha de monitorar latência como primeiro direcionador reflete um risco de negócio central para a ASIS: o fluxo de ingestão de documentos fiscais opera com **alta intensidade** (lotes de milhares de documentos) e exige **precisão de atendimento** — qualquer degradação no tempo de resposta impacta diretamente a cadência de processamento dos clientes e pode gerar gargalos em cascata nos módulos subsequentes (EFD, NCM, etc.).

A decisão de implementar limiares de alerta a 650 ms (30% acima do SLA de 500 ms) decorre da necessidade de **detecção proativa de degradação**, permitindo que camadas de orquestração e futuros modelos de aprendizado de máquina reajam a tendências de lentidão antes que estas se tornem violações de SLA reais. O desenho assíncrono do processamento (geração de `tracking_id` como primeira resposta, processamento em background) é uma decisão arquitetural que separa a latência percebida pelo cliente da complexidade real de auditoria.

Impacto na Qualidade — Aspectos Positivos e Negativos Mapeados

Aspectos positivos:

- O fluxo de ingestão mantém latência consistente (~373 ms de média), muito abaixo do SLA, preservando margem para crescimento de volume e intensidade de processamento.
- A estrutura de métricas (min, max, média, P90) fornece base quantitativa para dashboards operacionais e futura calibração de auto-scaling ou modelos de aprendizado de máquina para detecção de anomalias de desempenho.
- A separação entre resposta rápida (ID de processo) e processamento assíncrono é uma decisão arquitetural que protege a experiência do cliente independentemente da complexidade do lote fiscal.
- O mecanismo de alertas automatizados (limiar de 650 ms) permite ação corretiva antes que a degradação afete o fluxo fim a fim de auditoria.

Aspectos negativos / fora do foco atual:

- Os testes de latência foram executados em ambiente local (TestClient in-process), sem rede real nem contenção de recursos. Em produção, variáveis como latência de rede, contenção de CPU/memória e concorrência real podem alterar os resultados.
- Não foram testados cenários de degradação gradual (ex.: aumento progressivo de carga até ruptura) — esse aspecto é endereçado pelo Red Team (DN1).
- A persistência assíncrona simulada não reflete a complexidade de um storage real (banco de dados, fila de mensagens), o que pode adicionar latência não capturada nos testes atuais.
- Não há integração com ferramentas de APM (Application Performance Monitoring) em tempo real, limitando a visibilidade em ambientes distribuídos.

Justificativa de priorização: A latência de ingestão foi escolhida como primeiro direcionador por ser o atributo de qualidade com impacto mais direto na operação diária dos clientes ASIS e na intensidade do fluxo fiscal. Limitações de tempo e acesso à infraestrutura real impediram testes com rede e storage reais, ficando como recomendação para fases futuras.

1.2 DN2 (Blue Team) - Integridade dos Dados Fiscais na Ingestão

Objetivo

Garantir a validação rigorosa de payloads fiscais **antes do processamento**, evitando erros

de integridade de dados na ingestão de documentos fiscais. Este direcionador assegura que apenas lotes válidos sejam processados, protegendo a integridade dos dados da ASIS TaxTech.

Regras de Negócio

DN2.1 - Validação de Campos Obrigatórios

- A API deve rejeitar requisições com campos obrigatórios ausentes: `sessao_cliente`, `valor`, `destino`
- Retorna HTTP 400 com mensagem clara
- Garante rastreabilidade e roteamento correto das auditorias

DN2.2 - Validação de Valor Positivo

- A API deve rejeitar requisições onde `valor` ≤ 0
- Aceita valores inteiros e decimais positivos
- O tipo deve ser numérico (int ou float)
- Fundamenta-se na impossibilidade lógica de auditar lotes com volume zero ou negativo

DN2.3 & DN2.4 - Sem Processamento em Erro

- A API **NÃO** gera `tracking_id` para payloads inválidos
- A API **NÃO** inicia processamento assíncrono para payloads inválidos
- Validação ocorre **ANTES** de qualquer operação assíncrona (`await`)
- Evita alocação desnecessária de recursos e garante atomicidade

DN2.5 - Preservação de Contrato

- Payloads válidos mantêm o comportamento original da API
- Resposta preserva mesma estrutura: id, status, tracking_id, detalhes, etc.
- Headers obrigatórios (account-key, app-key) continuam sendo validados

Especificação Técnica

Endpoint afetado: `POST /api/v1/auditar`

Validações implementadas em src/core/mock_api.py:

```
# DN2.1: Verificação de campos (sessao_cliente, valor, destino)
if sessao is None or modulo_fiscal is None or volume_docs is None:
    raise HTTPException(status_code=400, detail="Parâmetros obrigatórios a

# DN2.2: Validação de tipo e magnitude numérica
if not isinstance(volume_docs, (int, float)) or volume_docs <= 0:
    raise HTTPException(status_code=400, detail="Valor deve ser um número

# DN2.4 & DN2.5: Processamento e geração de tracking_id apenas após valida
await asyncio.sleep(random.uniform(0.1, 0.5))
return { "tracking_id": str(uuid.uuid4()), ... }
```

Metodologia de Testagem (TDD)

A validação foi conduzida através de abordagem Test Data Driven (TDD) com 17 testes automatizados em `src/tests/test_mock_api.py`:

TestDN2IntegrityValidation (14 testes)

- 3 testes: Campos obrigatórios ausentes (sessao_cliente, valor, destino)
- 4 testes: Validação de valor (negativo, zero, zero decimal, tipo string)
- 4 testes: Payloads válidos (inteiro, decimal, valor mínimo, valor máximo)
- 3 testes: Robustez (múltiplos campos ausentes, payload vazio, campos extras)

TestDN2ContractValidation (3 testes)

- Endpoint e status code HTTP
- Headers obrigatórios ainda validados
- Estrutura completa de resposta preservada

Resultados Obtidos

A suíte de testes validou a implementação do DN2 com os seguintes indicadores:

- **Status Final:** APROVADO □
- **Taxa de Cobertura:** 100% (17/17 testes passando)
- **Cenários Testados:**
  - 7 cenários de rejeição (erro HTTP 400)
  - 7 cenários de aceitação (sucesso HTTP 201)
  - 3 validações de contrato
- **Verificações de Negócio:**
  - DN2.1: Campos obrigatórios → 3 testes □
  - DN2.2: Valor > 0 → 4 testes □
  - DN2.3: HTTP 400 para inválidos → 7 testes □
  - DN2.4: Sem tracking_id em erro → 7 testes □
  - DN2.5: Contrato preservado → 6 testes □

Execução dos Testes

```
# Instalar dependências
pip install -r requirements.txt

# Executar todos os testes DN2
pytest src/tests/blue_team/validation.py -v

# Executar apenas testes de integridade
pytest src/tests/blue_team/validation.py::TestDN2IntegrityValidation -v

# Executar apenas testes de contrato
pytest src/tests/blue_team/validation.py::TestDN2ContractValidation -v
```

Decisão Arquitetural e Contexto de BD/DN

A validação rigorosa de payloads antes do processamento é uma decisão arquitetural derivada diretamente de um risco de negócio crítico: em um sistema de auditoria fiscal com **alto volume e intensidade de ingestão**, permitir que dados malformados avancem no pipeline consome recursos computacionais desnecessariamente e pode gerar resultados incorretos que comprometam a **precisão do atendimento** ao cliente. O padrão *fail-fast* (rejeitar antes de processar) foi adotado para proteger a integridade do fluxo fim a fim.

A abordagem de validar **antes** de qualquer operação assíncrona (`await`) reflete uma escolha arquitetural consciente: garantir atomicidade e evitar que recursos de processamento, rastreio e armazenamento sejam alocados para dados que não atendem aos requisitos mínimos de negócio. Essa estratégia também cria uma base sólida para futuras camadas de **aprendizado de máquina**, já que modelos de detecção de anomalias ou classificação fiscal dependem de dados de entrada com integridade garantida.

Impacto na Qualidade — Aspectos Positivos e Negativos Mapeados

Aspectos positivos:

- **Confiabilidade do fluxo fiscal:** Rejeição proativa de dados malformados impede que documentos inválidos contaminem o pipeline de auditoria, preservando a precisão de atendimento.
- **Economia de recursos:** Validação anterior ao processamento assíncrono evita alocação desnecessária de CPU, memória e operações de I/O para dados que seriam

descartados.
- **Rastreabilidade consistente:** Apenas lotes válidos recebem `tracking_id`, garantindo que cada ID gerado corresponda a um processamento real — essencial para reconciliação, billing e análises de negócio.
- **Base para automação e ML:** A qualidade garantida dos dados de entrada cria fundação confiável para modelos futuros de classificação de lotes, detecção de anomalias fiscais e aprendizado de máquina sobre padrões de ingestão.
- **Manutenibilidade:** 17 cenários de teste documentados cobrem as regras de negócio de forma reprodutível.

Aspectos negativos / fora do foco atual:

- Não foram implementadas validações semânticas mais profundas (ex.: verificar se o `destino` corresponde a um módulo fiscal real cadastrado na ASIS), apenas validações estruturais e de tipo.
- Valores extremos porém tecnicamente válidos (ex.: `valor = 1e308`) não são barrados pela validação atual — esse risco é explorado pelo Red Team (DN2.R2).
- A validação de `destino` aceita qualquer string não vazia, sem conferir contra uma lista de módulos permitidos, o que pode gerar processamento de lotes para destinos inexistentes.
- Não há mecanismo de rate limiting na camada de validação; requisições inválidas em alta frequência ainda consomem ciclos de validação, o que é mitigado pela camada de disponibilidade (DN3).

Justificativa de priorização: A integridade de dados fiscais foi priorizada como segundo direcionador por ser pré-requisito para todos os demais: sem dados válidos, métricas de latência, disponibilidade e não-repúdio perdem significado. Limitações de tempo impediram a implementação de validações semânticas mais profundas (ex.: lista de módulos válidos, tetos fiscais), ficando como recomendação para evolução futura.

1.3 DN3 (Blue Team) - Disponibilidade e Resiliência da API

Objetivo

Garantir a **disponibilidade contínua e resiliência** da API de auditoria fiscal da ASIS TaxTech, implementando mecanismos de proteção contra sobrecarga, falhas em cascata e reprocessamentos indevidos. Este direcionador assegura que o sistema Kolossus permaneça operacional sob condições adversas, com degradação graciosa em vez de falha total.

Regras de Negócio

DN3.1 - Rate Limiting (Controle de Taxa)

- A API deve limitar requisições por `account-key` dentro de uma janela deslizante de 60 segundos
- Máximo de 10 requisições por janela temporal por conta
- Requisições excedentes retornam HTTP 429 (Too Many Requests)
- O isolamento entre contas garante que um cliente não afete outro

DN3.2 - Circuit Breaker (Disjuntor de Circuito)

- Após 5 falhas consecutivas, o circuit breaker **abre** e bloqueia novas requisições
- O circuito transita por 3 estados: `CLOSED` → `OPEN` → `HALF_OPEN`
- Após período de recuperação (30s), permite uma requisição de teste (`HALF_OPEN`)
- Um sucesso em `HALF_OPEN` reseta o circuito para `CLOSED`

DN3.3 - Health Check (Monitoramento de Saúde)

- Endpoint `GET /api/v1/health` retorna o estado operacional da API
- Inclui: uptime, total de requisições, taxa de erro, estado do circuit breaker
- Status `healthy` quando taxa de erro < 30%; `degraded` quando ≥ 30%
- Não exige autenticação (monitoramento externo)

DN3.4 - Idempotência de Requisições

- Requisições com header `idempotency-key` retornam o mesmo resultado se duplicadas
- A primeira execução processa normalmente; duplicatas retornam o resultado armazenado
- Campo `idempotent_replay: true` indica que é uma resposta cacheada
- Sem `idempotency-key`, cada requisição gera resultado novo

DN3.5 - Degradação Graciosa

- Quando o circuit breaker está aberto, a API retorna HTTP 503 com resposta de fallback
- A resposta degradada preserva os dados do cliente e informa `retry_after_seconds`
- Status 207 (Multi-Status) com diagnóstico "Degradado" e motivo detalhado

DN3.6 - Alertas de Disponibilidade

- Alerta emitido quando taxa de erro excede 30% (limiar padrão DN1)
- Alerta emitido quando circuit breaker está no estado `OPEN`
- Alerta emitido quando restam menos de 30% das requisições no rate limit
- Todos os alertas seguem o padrão `ALERTA DISPONIBILIDADE: [mensagem]`

Especificação da API — Endpoints de Resiliência

Endpoint principal: `POST /api/v1/auditar/resilient`

A API estende o endpoint original (`/api/v1/auditar`) com uma camada completa de resiliência. Os mesmos campos de entrada são utilizados:

- `sessao_cliente`: Identificador único da sessão de auditoria.
- `valor`: Volume de documentos fiscais submetidos no lote.
- `destino`: Módulo fiscal de destino (ex: `EFD_ICMS_IPI`, `NCM_AUDIT`).

Headers obrigatórios: `account-key`, `app-key` **Header opcional:** `idempotency-key` (chave de idempotência)

A resposta inclui todos os campos originais (DN2) mais o bloco `availability`:


```
{
  "id": 25000,
  "status": 201,
  "tracking_id": "uuid-v4",
  "arquivold": 22500,
  "diagnostico": "Conforme",
  "detalhes": { "cliente": "...", "volume_processado": 3200, "modulo": "..
  "availability": {
    "circuit_state": "CLOSED",
    "rate_limit_remaining": 8,
    "idempotent_replay": false,
    "alert": null
  }
}
```


Endpoint de monitoramento: `GET /api/v1/health`

```
{
  "status": "healthy",
  "uptime_seconds": 120.5,
  "total_requests": 45,
  "total_errors": 2,
  "error_rate_percent": 4.44,
  "timestamp": "2026-02-12T12:00:00.000000+00:00",
  "version": "1.2.0",
  "circuit_breaker": "CLOSED",
  "availability_alert": null,
  "circuit_alert": null
}
```


Mapa de Direcionadores de Negócio — Blue Team (DN3)

Foram implementados 6 direcionadores que garantem a disponibilidade e resiliência completas:

ID	Direcionador de Negócio	SLA / Critério de Aceite	Status
DN3.1	**Rate Limiting**	Máximo 10 req/min por account-key; isolamento entre contas; HTTP 429 ao exceder.	**PASS**
DN3.2	**Circuit Breaker**	Abertura após 5 falhas; transição CLOSED→OPEN→HALF_OPEN; reset com sucesso.	**PASS**
DN3.3	**Health Check**	Endpoint público retorna status, uptime, taxa de erro e estado do circuito.	**PASS**
DN3.4	**Idempotência**	Requisições duplicadas com mesma chave retornam resultado idêntico sem reprocessar.	**PASS**
DN3.5	**Degradação Graciosa**	Fallback com HTTP 503/207, dados do cliente preservados, `retry_after` informado.	**PASS**
DN3.6	**Alerta de Disponibilidade**	Alertas automáticos para taxa de erro >30%, circuit breaker aberto e rate limit próximo.	**PASS**

Implementação Técnica

Módulo de disponibilidade (`src/core/availability.py`):

```python
# DN3.1: Rate Limiter com janela deslizante
class RateLimiter:
    def is_allowed(account_key) -> bool:
    def remaining(account_key) -> int:

# DN3.2: Circuit Breaker com 3 estados
class CircuitBreaker:
    def allow_request() -> bool:
    def record_success():
    def record_failure():

# DN3.3: Health Check com métricas
class HealthChecker:
    def record_request(success: bool):
    def get_status() -> dict:

# DN3.4: Store de idempotência
class IdempotencyStore:
    def generate_key(account_key, payload) -> str:
    def get(key) -> Optional[dict]:
    def store(key, response):

# DN3.5: Degradação graciosa
def build_degraded_response(payload, reason) -> dict:

# DN3.6: Alertas de disponibilidade
def check_availability_alert(health) -> Optional[str]:
def check_circuit_alert(cb) -> Optional[str]:
def check_rate_limit_alert(rl, account_key) -> Optional[str]:
```

Metodologia de Testagem (Blue Team — TDD)

A aferição foi conduzida utilizando abordagem Test Data Driven (TDD) através do script `src/tests/blue_team/availability.py`.

Cada cenário de teste é executado 3 vezes (iterações parametrizadas via `@pytest.mark.parametrize`) para garantir consistência e reprodutibilidade dos resultados.

Massa de testes não-funcionais: Os payloads de teste simulam sessões reais de auditoria fiscal com volumes variados (`PAYLOAD_VALIDO`: 3200.00 docs para EFD_ICMS_IPI; `PAYLOAD_ALTERNATIVO`: 8100.75 docs para NCM_AUDIT).

Versão da API testada: v1.2.0 — `POST /api/v1/auditar/resilient` | `GET /api/v1/health`

Estrutura dos testes (8 classes, 36 cenários × 3 iterações = 108 execuções):

Classe de Teste	Cenários	Direcionador
`TestDN3RateLimiting`	5 testes × 3 iter	DN3.1
`TestDN3CircuitBreaker`	5 testes × 3 iter	DN3.2
`TestDN3HealthCheck`	6 testes × 3 iter	DN3.3
`TestDN3Idempotencia`	5 testes × 3 iter	DN3.4
`TestDN3DegradacaoGraciosa`	4 testes × 3 iter	DN3.5
`TestDN3Alertas`	7 testes × 3 iter	DN3.6
`TestDN3ContratoAPI`	4 testes × 3 iter	Contrato

Resultados Obtidos

A suíte de testes validou a implementação do DN3 com os seguintes indicadores:

- **Status Final:** APROVADO □
- **Taxa de Cobertura:** 100% (36 cenários × 3 iterações = 108 execuções passando)
- **Cenários Testados:**

- 15 cenários de rate limiting (DN3.1)
- 15 cenários de circuit breaker (DN3.2)
- 18 cenários de health check (DN3.3)
- 15 cenários de idempotência (DN3.4)
- 12 cenários de degradação graciosa (DN3.5)
- 21 cenários de alertas de disponibilidade (DN3.6)
- 12 cenários de contrato da API

- **Verificações de Negócio:**
  - DN3.1: Rate limiting por conta → 5 testes □
  - DN3.2: Circuit breaker 3 estados → 5 testes □
  - DN3.3: Health check público → 6 testes □
  - DN3.4: Idempotência de requisições → 5 testes □
  - DN3.5: Degradação graciosa → 4 testes □
  - DN3.6: Alertas de disponibilidade → 7 testes □
  - Contrato API preservado → 4 testes □

Execução dos Testes

```
# Instalar dependências
pip install -r requirements.txt

# Executar todos os testes DN3 (Blue Team)
pytest src/tests/blue_team/availability.py -v

# Executar apenas testes de rate limiting
pytest src/tests/blue_team/availability.py::TestDN3RateLimiting -v

# Executar apenas testes de circuit breaker
pytest src/tests/blue_team/availability.py::TestDN3CircuitBreaker -v

# Executar apenas testes de health check
pytest src/tests/blue_team/availability.py::TestDN3HealthCheck -v

# Executar apenas testes de idempotência
pytest src/tests/blue_team/availability.py::TestDN3Idempotencia -v

# Executar apenas testes de degradação graciosa
pytest src/tests/blue_team/availability.py::TestDN3DegradacaoGraciosa -v

# Executar apenas testes de alertas
pytest src/tests/blue_team/availability.py::TestDN3Alertas -v

# Executar apenas testes de contrato
pytest src/tests/blue_team/availability.py::TestDN3ContratoAPI -v
```

Decisão Arquitetural e Contexto de BD/DN

A implementação de mecanismos de disponibilidade e resiliência reflete uma decisão arquitetural advinda diretamente dos riscos de negócio identificados: o sistema de auditoria fiscal processa fluxos de **alta intensidade** e qualquer indisponibilidade impacta diretamente a cadeia de atendimento dos clientes ASIS. A combinação de rate limiting, circuit breaker, health check e idempotência forma uma arquitetura de defesa em profundidade que protege não apenas o serviço imediato, mas também sustenta a **precisão do atendimento** sob condições adversas.

A escolha de implementar singletons de estado (`RateLimiter`, `CircuitBreaker`, `HealthChecker`, `IdempotencyStore`) é uma decisão de BD/DN: centralizar o controle de disponibilidade permite que o sistema reaja de forma coordenada a eventos de degradação, evitando decisões conflitantes entre componentes. O design de métricas em tempo real (`uptime`, `error_rate_percent`, `circuit_breaker`) não apenas serve para observabilidade operacional, mas cria uma **base de dados estruturada para futuros modelos de aprendizado de máquina** capazes de prever degradações, ajustar auto-scaling e otimizar roteamento de tráfego automaticamente.

Impacto na Qualidade — Aspectos Positivos e Negativos Mapeados

Aspectos positivos:

- **Disponibilidade do fluxo fiscal:** Rate limiting e circuit breaker previnem sobrecarga e falhas em cascata, garantindo continuidade operacional mesmo sob picos de demanda.
- **Resiliência com dignidade:** Degradação graciosa garante operação parcial (com informação útil ao cliente, como `retry_after`) em vez de falha total — decisão que preserva a confiança do parceiro no fluxo de auditoria.
- **Observabilidade para decisões inteligentes:** Health check e alertas permitem monitoramento contínuo e proativo, com dados suficientes para alimentar painéis operacionais e futuros modelos de ML de detecção de anomalias.
- **Precisão de atendimento:** Idempotência impede reprocessamentos indevidos e duplicidades fiscais, garantindo que cada operação de auditoria seja contabilizada com exatidão.
- **Escalabilidade controlada:** Isolamento de rate limit por conta permite crescimento do número de clientes sem degradação cruzada.
- **Integração facilitada:** Informações de `retry_after` e `rate_limit_remaining` nas respostas permitem que clientes ajustem automaticamente seu ritmo de envio.

Aspectos negativos / fora do foco atual:

- O `RateLimiter` utiliza um dicionário em memória sem política de expiração (TTL/eviction), o que representa risco de crescimento ilimitado de memória em ambientes com muitas contas — mapeado e explorado pelo Red Team (DN3.R6).
- O `IdempotencyStore` apresenta comportamento de limitação implícita que pode não registrar todos os replays, reduzindo rastreabilidade em cenários de alta volumetria — explorado pelo Red Team (DN3.R8).
- A sinalização de `idempotent_replay` apresenta inconsistência em cenários de colisão de chaves — explorado pelo Red Team (DN3.R3).
- Não há persistência de estado entre reinicializações da API: se o serviço reiniciar, contadores de rate limit, estado do circuit breaker e store de idempotência são perdidos.
- Os testes foram executados em ambiente single-node; em cenários distribuídos (múltiplas instâncias), os mecanismos de estado precisariam de um store compartilhado (Redis, por exemplo).

Justificativa de priorização: A disponibilidade foi priorizada como terceiro direcionador por representar a camada de proteção do fluxo fim a fim: mesmo que dados e latência estejam corretos, indisponibilidade invalida toda a cadeia. A escolha por armazenamento em memória (vs. Redis/externo) foi uma decisão consciente para viabilizar execução local e testes reprodutíveis dentro das limitações de infraestrutura e tempo do projeto, com a arquitetura preparada para migração futura.

1.4 DN4 (Blue Team) - Não-Repúdio de Transações de Auditoria Fiscal

Objetivo

Garantir que todas as transações de auditoria fiscal sejam realizadas com **mecanismos criptográficos de não-repúdio**, impedindo que qualquer das partes negue a realização de uma transação. Este direcionador subsidia as áreas de segurança e suporte aos clientes da ASIS TaxTech, fornecendo evidências criptográficas de **quem** realizou a transação, **o quê** foi transacionado, e **quando** ocorreu.

Especificação da API — Endpoint de Não-Repúdio

Endpoint: `POST /api/v1/auditar/nr`

A API estende o endpoint original (`/api/v1/auditar`) com uma camada completa de não-repúdio. Os mesmos campos de entrada são utilizados:

- `sessao_cliente`: Identificador único da sessão de auditoria.
- `valor`: Volume de documentos fiscais submetidos no lote.
- `destino`: Módulo fiscal de destino (ex: `EFD_ICMS_IPI`, `NCM_AUDIT`).

Headers obrigatórios: `account-key`, `app-key`

A resposta inclui todos os campos originais (DN2) mais o bloco `non_repudiation`:

```
{
  "id": 25000,
  "status": 201,
  "tracking_id": "uuid-v4",
  "arquivold": 22500,
  "diagnostico": "Conforme",
  "detalhes": { "cliente": "...", "volume_processado": 2500, "modulo": "..
  "non_repudiation": {
    "signature": "hmac-sha256-hex-64-chars",
    "timestamp": "2026-02-12T12:00:00.000000+00:00",
    "actor": "sha256-hex-64-chars",
    "receipt": {
      "signature": "...",
      "timestamp": "...",
      "actor": "...",
      "tracking_id": "...",
      "payload_hash": "sha256-hex-64-chars"
    },
    "log_position": 0,
    "log_entry_hash": "sha256-hex-64-chars",
    "alert": null,
    "chain_alert": null
  }
}
```

Endpoints auxiliares de auditoria:

- `GET /api/v1/audit-log` — Retorna o log completo com status de integridade da cadeia de hash.
- `GET /api/v1/audit-log/{position}` — Retorna uma entrada específica do log.

Mapa de Direcionadores de Negócio — Blue Team (DN4)

Foram implementados 6 direcionadores que garantem o não-repúdio completo das transações:

ID	Direcionador de Negócio	SLA / Critério de Aceite	Status
DN4.1	**Assinatura Digital (HMAC-SHA256)**	100% das transações válidas retornam assinatura HMAC-SHA256 verificável sobre payload + timestamp.	**PASS**
DN4.2	**Carimbo Temporal Imutável**	100% das transações incluem timestamp ISO 8601 server-side; tolerância ≤ 2s do horário real.	**PASS**
DN4.3	**Cadeia de Autenticação**	Toda resposta registra o `actor` (SHA-256 da account-key), vinculando identidade à transação.	**PASS**
DN4.4	**Recibo Digital Completo**	Respostas válidas incluem recibo com: `signature`, `timestamp`, `actor`, `tracking_id`, `payload_hash`.	**PASS**
DN4.5	**Integridade do Log de Auditoria**	Log com cadeia de hash (hash chain); cada entrada referencia `prev_hash` da anterior. Cadeia verificável.	**PASS**
DN4.6	**Alerta de Violação de Não-Repúdio**	Sistema emite `ALERTA` quando campos de não-repúdio estão ausentes ou cadeia de hash comprometida. Limiar de 30% (padrão DN1).	**PASS**

Implementação Técnica

Módulo de não-repúdio (`src/core/non_repudiation.py`):

```
# DN4.1: Assinatura digital HMAC-SHA256
def sign_transaction(payload, timestamp, secret) -> str:
    message = json.dumps(payload, sort_keys=True) + "|" + timestamp
    return hmac.new(secret.encode(), message.encode(), hashlib.sha256).hex

# DN4.2: Carimbo temporal imutável (server-side, UTC)
def generate_timestamp() -> str:
    return datetime.now(timezone.utc).isoformat()

# DN4.3: Hash do ator autenticado
def hash_actor(account_key) -> str:
    return hashlib.sha256(account_key.encode()).hexdigest()

# DN4.4: Recibo digital completo
def build_receipt(signature, timestamp, actor, tracking_id, payload) -> di
    return {"signature": ..., "timestamp": ..., "actor": ...,
            "tracking_id": ..., "payload_hash": hash_payload(payload)}

# DN4.5: Log de auditoria com hash chain
class AuditLog:
    def append(self, receipt) -> dict:  # Adiciona entrada com prev_hash e
    def verify_chain(self) -> bool:      # Verifica integridade da cadeia

# DN4.6: Alertas de violação
def check_alert(receipt) -> Optional[str]:       # Campos ausentes no reci
def check_chain_alert(log) -> Optional[str]:     # Cadeia de hash comprome
```

Metodologia de Testagem (Blue Team — TDD)

A aferição foi conduzida utilizando abordagem Test Data Driven (TDD) através do script `src/tests/blue_team/non_repudiation.py`.

Cada cenário de teste é executado 3 vezes (iterações parametrizadas via `@pytest.mark.parametrize`) para garantir consistência e reprodutibilidade dos resultados.

Massa de testes não-funcionais: Os payloads de teste simulam sessões reais de auditoria fiscal com volumes variados (`PAYLOAD_VALIDO`: 2500.00 docs para EFD_ICMS_IPI; `PAYLOAD_ALTERNATIVO`: 7500.50 docs para NCM_AUDIT).

Versão da API testada: v1.1.0 — `POST /api/v1/auditar/nr`

Estrutura dos testes (7 classes, 24 cenários × 3 iterações = 72 execuções):

Classe de Teste	Cenários	Direcionador
`TestDN4AssinaturaDigital`	4 testes × 3 iter	DN4.1
`TestDN4CarimboTemporal`	3 testes × 3 iter	DN4.2
`TestDN4CadeiaAutenticacao`	3 testes × 3 iter	DN4.3
`TestDN4ReciboDigital`	4 testes × 3 iter	DN4.4
`TestDN4LogAuditoria`	4 testes × 3 iter	DN4.5
`TestDN4Alertas`	5 testes × 3 iter	DN4.6
`TestDN4ContratoAPI`	3 testes × 3 iter	Contrato
`TestDN4ConsultaLog`	3 testes × 3 iter	DN4.5

Resultados Obtidos

A suíte de testes validou a implementação do DN4 com os seguintes indicadores:

- **Status Final:** APROVADO
- **Taxa de Cobertura:** 100% (29 cenários × 3 iterações = 87 execuções passando)
- **Cenários Testados:**
  - 12 cenários de assinatura e criptografia (DN4.1)
  - 9 cenários de carimbo temporal (DN4.2)
  - 9 cenários de cadeia de autenticação (DN4.3)
  - 12 cenários de recibo digital (DN4.4)
  - 12 cenários de integridade do log (DN4.5)
  - 15 cenários de alertas (DN4.6)
  - 9 cenários de contrato da API
  - 9 cenários de consulta do log
- **Verificações de Negócio:**
  - DN4.1: Assinatura HMAC-SHA256 → 4 testes
  - DN4.2: Timestamp ISO 8601 ≤ 2s → 3 testes
  - DN4.3: Actor SHA-256 → 3 testes
  - DN4.4: Recibo completo → 4 testes
  - DN4.5: Hash chain íntegra → 4 testes
  - DN4.6: Alertas de violação → 5 testes
  - Contrato API preservado → 3 testes
  - Consulta de log → 3 testes

Execução dos Testes

```
# Instalar dependências
pip install -r requirements.txt

# Executar todos os testes DN4 (Blue Team)
pytest src/tests/blue_team/non_repudiation.py -v

# Executar apenas testes de assinatura digital
pytest src/tests/blue_team/non_repudiation.py::TestDN4AssinaturaDigital -v

# Executar apenas testes de carimbo temporal
pytest src/tests/blue_team/non_repudiation.py::TestDN4CarimboTemporal -v

# Executar apenas testes de cadeia de autenticação
pytest src/tests/blue_team/non_repudiation.py::TestDN4CadeiaAutenticacao -

# Executar apenas testes de recibo digital
pytest src/tests/blue_team/non_repudiation.py::TestDN4ReciboDigital -v

# Executar apenas testes de log de auditoria
pytest src/tests/blue_team/non_repudiation.py::TestDN4LogAuditoria -v

# Executar apenas testes de alertas
pytest src/tests/blue_team/non_repudiation.py::TestDN4Alertas -v

# Executar apenas testes de contrato
pytest src/tests/blue_team/non_repudiation.py::TestDN4ContratoAPI -v
```

Decisão Arquitetural e Contexto de BD/DN

A implementação de não-repúdio decorre de um risco de negócio central em auditoria fiscal: a necessidade de **evidência irrefutável** de quem realizou cada transação, o quê foi processado e quando ocorreu. Em sistemas com **fluxos de alta intensidade** e múltiplos atores (clientes, auditores, sistemas automatizados), a ausência de mecanismos criptográficos de não-repúdio cria lacunas de responsabilidade que podem comprometer disputas fiscais, processos de compliance e a confiança dos clientes no atendimento da ASIS.

A escolha de HMAC-SHA256 como algoritmo de assinatura, combinada com hash chain no log de auditoria, é uma decisão arquitetural que equilibra **segurança criptográfica** com **eficiência computacional** — essencial para não comprometer a latência do fluxo de ingestão (DN1). O encadeamento de hashes no log cria uma trilha de auditoria com integridade verificável retroativamente, fornecendo base sólida para futuros sistemas de **aprendizado de máquina** em detecção de anomalias transacionais e para conformidade regulatória automatizada.

Impacto na Qualidade — Aspectos Positivos e Negativos Mapeados

Aspectos positivos:

- **Não-repúdio verificável:** Evidência criptográfica irrefutável de cada transação fiscal, vinculando identidade (`actor`), conteúdo (`payload_hash`) e momento (`timestamp`) — essencial para resolução de disputas e compliance.
- **Integridade do fluxo de auditoria:** A hash chain no log garante que qualquer adulteração retroativa seja detectada automaticamente, preservando a confiabilidade do histórico de processamento.
- **Precisão de atendimento:** O recibo digital completo serve como prova de processamento para o cliente, eliminando ambiguidades sobre o que foi auditado e quando.
- **Base para automação de compliance:** A estrutura de assinatura e log encadeado cria fundação para futuros sistemas de ML que podem detectar padrões anômalos de transações, tentativas de fraude e desvios regulatórios.
- **Segurança por design:** A assinatura HMAC-SHA256 impede adulteração pós-processamento sem acesso à chave secreta, protegendo a integridade das evidências fiscais.

- **Monitoramento proativo:** O sistema de alertas detecta campos de não-repúdio ausentes e cadeias de hash comprometidas antes que o impacto se propague.

Aspectos negativos / fora do foco atual:

- O mecanismo de autenticação atual (account-key/app-key via header) não implementa verificação criptográfica real de identidade — é uma simulação. Em produção, seria necessário autenticação forte (ex.: mTLS, OAuth 2.0 com assinatura de request).
- Não há proteção anti-replay nativa: um atacante pode reenviar requisições idênticas sem bloqueio — risco explorado pelo Red Team (DN4.R1).
- O log de auditoria é acessível sem restrição de escopo (qualquer account-key pode ler todo o log), representando risco de vazamento de informações entre tenants — explorado pelo Red Team (DN4.R4).
- A chave secreta de HMAC é fixa e compartilhada; em produção, cada tenant deveria ter chaves rotativas com gestão adequada de ciclo de vida.
- O armazenamento do log é em memória, sem persistência entre reinicializações, o que limita a auditabilidade em cenários reais.

Justificativa de priorização: O não-repúdio foi priorizado como quarto direcionador por completar a cadeia de confiabilidade: latência garante velocidade, integridade garante correção, disponibilidade garante acesso, e não-repúdio garante responsabilidade. Limitações de tempo e escopo impediram a implementação de autenticação forte real, anti-replay e persistência de log, ficando como recomendações para evolução — decisões tomadas conscientemente para entregar evidência funcional dos mecanismos criptográficos dentro do prazo.

2.1 DN1 (Red Team) - Testes Adversariais de Latência (SLA)

Objetivo do Ativo Adversarial

Validar se a API de auditoria resiste a cenários de **estresse e carga** que visam **quebrar o SLA de latência** (DN1). O Red Team tenta degradar a resposta para além de 500 ms (alvo) e 650 ms (limite de alerta), de forma a expor fragilidades que o Blue Team não exercita.

Critérios de Ataque

- **SLA alvo:** ≤ 500 ms (90% das requisições), conforme Blue Team.
- **Limite de alerta:** 650 ms (30% acima do SLA). Qualquer resposta acima disso conta como violação.
- **Sucesso do Red Team:** se ≥ 10% das requisições em um cenário ultrapassarem 650 ms (ou a média/p95 estourar), o teste **falha** e o ativo é considerado vulnerável ao cenário.

Bateria de Testes e Evolução da Carga

A suíte em `src/tests/red_team/response_time_red.py` foi evoluída em etapas, com **aumento progressivo da carga** até tentar quebrar o SLA. Abaixo, o que cada teste faz e os parâmetros usados na última versão:

Teste	Descrição	Carga (versão final)
Baseline	Uma requisição válida; garante que o ambiente não já está lento.	1 req
Burst sequencial	Muitas requisições leves em sequência para ver degradação por volume.	**600 req**
Payload pesado	Um único JSON com milhares de campos extras (schema pollution) para estressar parsing.	**70.000 campos × 300 chars** (~21 MB)
String gigante	Campo `destino` com string massiva para estressar memória e serialização.	**10 MB**
Burst concorrente	Requisições em paralelo (threads) para simular pico de tráfego.	**60 workers × 12 req = 720 req** simultâneas
Burst pesado	Sequência de requisições, cada uma com payload grande (volume + parsing).	**100 req × 40.000 campos × 120 chars** (~4,8 MB/req)

Evolução dos parâmetros (aumento progressivo):

- **Burst sequencial:** 30 → 100 → 250 → **600** requisições.
- **Payload pesado:** 5k×50 → 15k×100 → 35k×200 → **70k×300** (campos × caracteres).
- **String gigante:** 1 MB → 2 MB → 5 MB → **10 MB**.
- **Burst concorrente:** 30×6 (180) → **60×12 (720)**.
- **Burst pesado:** 50×20k×80 → **100×40k×120**.

Métricas e Logs

Cada cenário que gera múltiplos tempos (burst sequencial, concorrente e burst pesado) calcula e exibe:

- **min, max, média e p95** (percentil 95) em ms.
- **Conformidade:** quantidade e percentual de respostas com tempo ≤ 650 ms.

Exemplo de saída (burst):

```
[RED] Burst - resumo
      min=109.68ms | max=492.39ms | média=310.27ms | p95=472.2ms
      dentro do SLA (≤650ms): 100/100 (100.0%)
```

Resultados Obtidos

Em todas as rodadas, **todos os testes passaram**: a API manteve respostas dentro do limite de 650 ms e, nos cenários com múltiplas requisições, a conformidade permaneceu ≥ 90%. Ou seja, nas condições atuais (ambiente local, TestClient in-process), o **Red Team não conseguiu quebrar o SLA**; o ativo se mostrou resiliente à bateria aplicada.

- **Status final da suíte:** 6 testes, todos **PASSED**.
- **Interpretação:** O ativo está preparado para os cenários de estresse documentados; em ambientes mais limitados (menos CPU/memória) ou com rede real, a mesma bateria pode ser reexecutada para revalidar.

Síntese de Impacto (Positivos x Negativos) e Priorizações

Do ponto de vista de BD/DN, os testes adversariais de latência validam se a **decisão arquitetural de processamento assíncrono** resiste à intensidade real de uso e a cenários extremos que o Blue Team não cobre.

Aspectos positivos (para além do foco imediato):

- O fluxo de auditoria manteve latência dentro dos limites mesmo sob **carga extrema** (720 requisições concorrentes, payloads de 21 MB, 600 requisições sequenciais), demonstrando que a decisão de separar resposta rápida de processamento background é sólida para cenários de alta intensidade.

- O parsing de JSON mostrou resiliência a schema pollution (70.000 campos extras) e strings massivas (10 MB), indicando que o runtime FastAPI/Python consegue absorver payloads adversariais sem degradação catastrófica — informação relevante para dimensionamento de infraestrutura.
- A evolução progressiva dos parâmetros de carga documenta os limiares testados, criando uma referência quantitativa para futuras avaliações de capacidade e para modelos de aprendizado de máquina que possam prever saturação com base em padrões de tráfego.

Aspectos negativos / riscos mapeados:

- Todos os testes foram executados em ambiente local (TestClient in-process), onde a latência de rede é zero e os recursos são exclusivos do teste. Em ambientes reais com contenção de CPU, memória limitada e latência de rede, os resultados provavelmente seriam diferentes.
- Não foram testados cenários de degradação sustentada (ex.: carga alta mantida por minutos/horas), apenas rajadas pontuais. O comportamento sob carga constante pode revelar problemas de memory leak ou acúmulo de conexões.
- O Red Team não conseguiu quebrar o SLA neste ambiente, o que pode indicar tanto robustez real quanto limitação do ambiente de teste para representar condições adversas reais.
- Cenários de carga mista (requisições válidas e inválidas simultaneamente) não foram explorados, embora representem o padrão real de tráfego em produção.

Priorização sugerida:

1. **Revalidar em ambiente com rede real e contenção de recursos** — essencial para confirmar se a resiliência observada se mantém fora do ambiente ideal.
2. **Adicionar testes de carga sustentada** — complementar as rajadas com cenários de estresse prolongado para mapear comportamento ao longo do tempo.
3. **Combinar cenários de DN1 com DN3** — testar latência sob condições onde rate limiting e circuit breaker estejam ativos simultaneamente.

Execução

```
# Via Makefile (recomendado)
make stress

# Direto com pytest
.venv/bin/python -m pytest src/tests/red_team/response_time_red.py -v -s
```

Execução dos Testes (API real) — DN1

Os testes de latência podem rodar contra a **API real**. A forma mais prática é usar um arquivo **.env** na raiz do projeto: defina `RED_TEAM_API_URL=http://127.0.0.1:8000` (ou a URL da sua API), suba a API em outro terminal e rode o pytest normalmente:

```
# Terminal 1 — subir a API (a partir de src/)
uvicorn core.api:app --reload --port 8000

# Terminal 2 — com .env configurado, basta:
pytest src/tests/red_team/DN1_response_time_red.py -v -s
```

Sem `RED_TEAM_API_URL` no .env ou no shell, o script usa TestClient (in-process).

2.2 DN2 (Red Team) - Integridade dos Dados Fiscais na Ingestão

Objetivo do Ativo Adversarial

Esta seção apresenta os resultados da análise de segurança e robustez realizada pelo **Red Team** sobre o Direcionador de Negócio 2 (DN2 - Integridade de Dados). O objetivo é validar se o sistema de auditoria da **ASIS TaxTech** é capaz de resistir a entradas

malformadas ou extremas que visam comprometer a integridade dos dados fiscais.

Mapa de Direcionadores de Negócio - Red Team (DN2)

Foram implementados 5 direcionadores que testam os limites da lógica de validação:

ID	Direcionador Adversarial	Objetivo Técnico	Status
DN2.R1	**Script Injection**	Validar se o sistema trata payloads maliciosos (`<script>`) como texto comum.	**PASS**
DN2.R2	**Extreme Number Resilience**	Testar o comportamento da API com valores não finitos (`inf`, `nan`) ou astronômicos.	**FAIL**
DN2.R3	**Schema Pollution**	Verificar se o excesso de campos (5.000+) causa degradação de performance.	**PASS**
DN2.R4	**Strict Type Validation**	Tentar enviar estruturas complexas (listas) onde se espera um valor numérico.	**PASS**
DN2.R5	**Buffer Stability**	Enviar strings massivas (1MB) para testar limites de memória do endpoint.	**PASS**

Métodos de Testagem

O teste foi conduzido utilizando o framework **pytest-9.0.2** em ambiente **Python 3.13.3**.

- **Abordagem**: Os testes podem rodar de duas formas: (1) com **TestClient** (in-process), típico para CI; (2) contra a **API real** (servidor HTTP em execução), definindo a variável de ambiente `RED_TEAM_API_URL`.
- **Escopo**: Foco exclusivo na validação de entrada (Input Validation) antes da geração do `tracking_id` e do processamento assíncrono.

Execução dos Testes (API real) — DN2

Os testes adversariais de DN2 podem ser executados contra a **API real** (servidor já em execução), permitindo validar o comportamento em ambiente próximo ao de produção (rede, serialização HTTP, timeouts).

Forma recomendada: usar **.env** na raiz do projeto. Defina `RED_TEAM_API_URL=http://127.0.0.1:8000` no arquivo `.env` na raíz do projeto. Suba a API em outro terminal e rode o pytest:

```
# Terminal 1 — a partir de src/
uvicorn core.api:app --reload --host 0.0.0.0 --port 8000

# Terminal 2 — com .env configurado
pytest src/tests/red_team/DN2_validation_red.py -v -s
```

Sem `RED_TEAM_API_URL` no .env ou no shell, o script usa o **TestClient** (in-process).

Análise dos Resultados (Logs de Execução)

A execução automatizada (`validation_red.py`) coletou 7 itens e identificou vulnerabilidades críticas no tratamento de números:

Sucessos de Resiliência

- **Injeção de Scripts**: O sistema tratou tags HTML/JS como literais, impedindo ataques de Cross-Site Scripting (XSS).
- **Poluição de Schema**: A API processou 5.000 campos extras sem violar o SLA de tempo de resposta.
- **Tipagem**: Tentativas de enviar listas no campo `valor` foram barradas com HTTP 400.

Falhas Críticas Identificadas

O teste de **Resiliência Numérica** falhou em três cenários específicos:

1. **ValueError (inf/nan)**: O `json.encoder` padrão do Python não é compatível com valores `inf` e `nan` fora do padrão JSON (RFC 8259). Isso causou a interrupção do cliente de teste.
2. **Bypass de Regra (1e308)**: O sistema retornou **HTTP 201 Created** para um valor de 1×10^{308}, que embora positivo, é computacionalmente extremo e fiscalmente impossível. O sistema deveria ter retornado **HTTP 400**.

Síntese de Impacto (Positivos x Negativos) e Priorizações

Do ponto de vista de BD/DN, os testes adversariais de integridade validam se as decisões arquiteturais de validação *fail-fast* resistem a entradas maliciosas e extremas que o Blue Team não exercita, e se o fluxo de auditoria preserva a **precisão de atendimento** sob condições adversariais.

Aspectos positivos (para além do foco imediato):

- O sistema demonstrou **resiliência contra ataques de injeção** (XSS via `<script>`), tratando tags HTML/JS como literais — essencial para proteger o fluxo de dados fiscais contra vetores de ataque comuns em APIs web.
- A validação de tipagem funcionou corretamente para estruturas complexas (listas no campo `valor`), indicando que a camada de validação está preparada para dados inesperados de integrações automatizadas.
- O processamento de schema pollution (5.000 campos extras) não violou o SLA de tempo de resposta, mostrando que a arquitetura tolera payloads inflados sem degradação catastrófica — informação relevante para a intensidade do fluxo real, onde clientes podem enviar campos adicionais por erro de integração.
- A cobertura adversarial complementa diretamente o Blue Team (DN2), criando um mapa de riscos que permite ao parceiro visualizar tanto o que foi protegido quanto o que permanece exposto.

Aspectos negativos / riscos mapeados:

- **Vulnerabilidade a valores extremos (DN2.R2):** O sistema aceita `1e308` como valor válido — fiscalmente impossível, mas numericamente positivo. Essa lacuna pode gerar distorções em métricas de volume, dashboards de negócio e modelos de ML que dependam de valores plausíveis.
- **Falha com infinitos/NaN:** O `json.encoder` padrão do Python não suporta `inf`/`nan` (RFC 8259), causando interrupção do cliente. Embora seja uma limitação do runtime e não do sistema, a API deveria tratar esses valores antes que cheguem ao encoder.
- Não foram testados cenários de injeção SQL (embora a API atual não use banco relacional, é um vetor relevante para evolução futura).
- Não há validação semântica do campo `destino` contra uma lista de módulos fiscais reais da ASIS, o que permite processamento de lotes para destinos inexistentes.

Argumentação e Ações Corretivas

Para que este ativo de software evolua em alinhamento com os riscos de negócio identificados, as seguintes ações são recomendadas, ordenadas por prioridade:

1. **Validação de Finitude (alta prioridade):** Implementar `math.isfinite(valor)` na `mock_api.py` para rejeitar infinitos e NaN antes do processamento — correção pontual com impacto direto na precisão dos dados.
2. **Limites de Sanidade (alta prioridade):** Estabelecer um teto máximo para o campo `valor` condizente com a realidade de lotes fiscais da ASIS (ex.: volume máximo documentado por sessão de auditoria).
3. **Validação de módulos válidos (média prioridade):** Implementar lista de valores permitidos para `destino`, evitando processamento para módulos inexistentes.
4. **Proteção contra injeção SQL (baixa prioridade para escopo atual):** Adicionar sanitização preventiva para cenários de evolução com banco de dados relacional.

Justificativa de priorização: As correções 1 e 2 são rápidas e endereçam falhas identificadas pelo Red Team com impacto direto na integridade do fluxo fiscal. As demais são recomendações para evolução, limitadas pelo tempo disponível e pelo escopo de simulação (mock API sem banco real).

2.3 DN4 (Red Team) - Não-Repúdio sob Perspectiva Adversarial

Objetivo do Ativo Adversarial

Esta seção documenta a validação ofensiva do DN4 para medir resistência a ataques de **replay**, **impersonação via headers** e **exposição de trilha de auditoria**. O foco é verificar se os mecanismos de não-repúdio permanecem robustos quando um atacante tenta abusar de lacunas de autenticação e autorização.

Mapa de Direcionadores de Negócio - Red Team (DN4)

Foram implementados 5 direcionadores adversariais:

ID	Direcionador Adversarial	Objetivo Técnico	Status
DN4.R1	**Replay Blocking**	Reenviar payload idêntico em janela curta e validar bloqueio por anti-replay.	**FAIL**
DN4.R2	**Header Impersonation**	Forjar `account-key` de terceiro e validar rejeição de identidade não autenticada.	**FAIL**
DN4.R3	**Signature Binding**	Confirmar que assinatura HMAC não valida payload adulterado.	**PASS**
DN4.R4	**Audit Log Authorization**	Tentar ler `/api/v1/audit-log` com credenciais não relacionadas ao ator original.	**FAIL**
DN4.R5	**Tamper Detection**	Adulterar hash de entrada e validar detecção de quebra de cadeia.	**PASS**

Métodos de Testagem

- Script adversarial: `src/tests/red_team/DN4_non_repudiation_red.py`
- Abordagem: `FastAPI TestClient` (in-process) ou requisições HTTP contra a API real quando `RED_TEAM_API_URL` está definida.
- Escopo: endpoint `POST /api/v1/auditar/nr` e consultas `GET /api/v1/audit-log`. O teste R5 (chain tampering) só roda in-process.

Execução dos Testes (API real) — DN4

Com **.env** configurado (`RED_TEAM_API_URL` em `.env.example`), suba a API e rode o pytest. O teste **DN4.R5** (adulteração da cadeia de hash) é pulado nesse modo por depender de estado interno do processo.

```
# Com .env na raiz (RED_TEAM_API_URL=http://127.0.0.1:8000)
pytest src/tests/red_team/DN4_non_repudiation_red.py -v -s
```

Análise de Resultados

- **Pontos fortes**: HMAC está corretamente vinculado ao payload/timestamp; adulteração da hash chain é detectada com alerta.
- **Lacunas críticas**: ausência de anti-replay e ausência de controle real de autenticação/autorização baseado em `account-key`/`app-key`.

Síntese de Impacto (Positivos x Negativos) e Priorizações

Do ponto de vista de BD/DN, os testes adversariais de não-repúdio validam se as **decisões criptográficas e de auditoria** implementadas pelo Blue Team resistem a ataques realistas que exploram lacunas de autenticação, autorização e controle de replay — aspectos críticos para a confiabilidade do fluxo fiscal e a precisão do atendimento.

Aspectos positivos (para além do foco imediato):

- A **vinculação HMAC ao payload e timestamp** (DN4.R3 PASS) confirma que o mecanismo de assinatura é robusto: um atacante que adultere o conteúdo da transação não consegue gerar uma assinatura válida sem acesso à chave secreta. Isso sustenta a integridade do fluxo de auditoria fiscal como evidência.
- A **detecção de adulteração na hash chain** (DN4.R5 PASS) demonstra que o log de auditoria é resistente a modificações retroativas, preservando a cadeia de evidências para compliance, auditorias regulatórias e futuros modelos de aprendizado de máquina que analisem padrões transacionais ao longo do tempo.
- Os mecanismos criptográficos implementados (HMAC-SHA256, SHA-256 para actor e payload, hash chain) formam uma base de segurança que, embora não completa, já reduz significativamente o risco de repúdio em comparação com sistemas sem qualquer evidência criptográfica.

Aspectos negativos / riscos mapeados:

- **Ausência de anti-replay (DN4.R1 FAIL):** Um atacante pode reenviar requisições válidas sem bloqueio, o que em um cenário fiscal pode resultar em duplicação de auditorias, distorção de métricas e potenciais implicações regulatórias.
- **Header impersonation não bloqueado (DN4.R2 FAIL):** A ausência de autenticação forte permite que qualquer agente forje `account-key` de terceiros e realize transações em nome de outro cliente — risco crítico para a confiabilidade do campo `actor` e para a rastreabilidade do fluxo de atendimento.
- **Log de auditoria sem controle de acesso (DN4.R4 FAIL):** Qualquer credencial pode ler todo o log, expondo informações de outros tenants — risco de vazamento de dados fiscais entre clientes.
- As falhas identificadas são de **autenticação e autorização**, não de criptografia — indicando que a camada matemática está sólida, mas a camada de controle de acesso precisa de evolução.

Ações Corretivas Recomendadas

Ordenadas por prioridade e impacto no risco de negócio:

1. **Implementar autenticação forte (alta prioridade):** Substituir o mecanismo atual de `account-key`/`app-key` por assinatura de request, chave registrada e verificação server-side (ex.: HMAC de request, mTLS ou OAuth 2.0). Endereça DN4.R2.
2. **Adicionar proteção anti-replay (alta prioridade):** Nonce + janela temporal + armazenamento de IDs de idempotência, impedindo reenvio de requisições já processadas. Endereça DN4.R1.
3. **Restringir acesso ao log por tenant/ator (média prioridade):** Aplicar checagem de permissão nos endpoints `GET /api/v1/audit-log*`, garantindo que cada cliente acesse apenas suas próprias transações. Endereça DN4.R4.
4. **Rotação de chaves HMAC (recomendação futura):** Implementar gestão de ciclo de vida de chaves secretas, com rotação periódica e suporte a múltiplas chaves ativas para transição suave.

Justificativa de priorização: As ações 1 e 2 endereçam os riscos mais críticos identificados pelo Red Team (impersonação e replay), que comprometem diretamente a confiabilidade do não-repúdio em um contexto fiscal. A ação 3 protege dados entre tenants. A ação 4 é recomendação de maturidade para produção. Limitações de tempo, escopo do mock e acesso a infraestrutura de autenticação real definiram o recorte atual.

2.4 DN3 (Red Team) - Disponibilidade e Resiliência da API

Objetivo do Ativo Adversarial

Esta seção documenta a bateria de testes adversariais do **Red Team** sobre o Direcionador de Negócio 3 (DN3 - Disponibilidade e Resiliência), implementada em `src/tests/red_team/availability_red.py`. O foco é tensionar os mecanismos de **rate limiting**, **circuit breaker**, **health check** e **idempotência** — pilares de BD/DN que sustentam o fluxo de auditoria fiscal com alta intensidade, precisão de atendimento e decisões automáticas baseadas em dados (aprendizado de máquina/heurísticas futuras).

Em termos de BD/DN, estes testes avaliam se as escolhas arquiteturais (singletons de disponibilidade, store de idempotência sem TTL, monitoramento em tempo real, alertas) realmente protegem o negócio sob cenários extremos, preservando o fluxo fim a fim de auditoria mesmo sob ataques ou mau uso.

Mapa de Direcionadores de Negócio - Red Team (DN3)

Foram definidos cenários adversariais alinhados aos mecanismos de disponibilidade:

ID	Direcionador Adversarial	Objetivo Técnico	Status
DN3.R1	Rate Limit Bypass por Header Spoofing	Tentar burlar o controle de taxa variando `account-key` em rajadas de alta intensidade.	**PASS**
DN3.R2	Circuit Breaker Race Condition	Estressar o `CircuitBreaker` com chamadas concorrentes de sucesso/falha e validar consistência.	**PASS**
DN3.R3	Idempotency Key Collision com Payload Diferente	Enviar mesma `idempotency-key` com payloads distintos para validar sinalização correta de replay.	**FAIL**
DN3.R4	Health Check Information Leakage	Verificar se o `GET /health` não vaza informações sensíveis de infraestrutura/stack interna.	**PASS**
DN3.R5	Slowloris / Connection Exhaustion (simples)	Simular requisições “lentas” para testar degradação controlada sem colapso nem cascata de erros.	**PASS**
DN3.R6	Rate Limit Memory Exhaustion	Inflar o dicionário interno do rate limiter com chaves gigantes para medir risco de consumo de RAM.	**PASS**
DN3.R8	Idempotency Store Unbounded Growth	Forçar muitas chaves de idempotência únicas e observar crescimento (ou não) do `IdempotencyStore`.	**FAIL**

Métodos de Testagem e Execução

- **Script**: `src/tests/red_team/DN3_availability_red.py`
- **Ferramenta**: `pytest` com `FastAPI TestClient` (in-process) ou requisições HTTP quando `RED_TEAM_API_URL` está definida.
- Em modo API real, os testes **DN3.R2**, **DN3.R6** e **DN3.R8** são pulados (dependem de estado interno: circuit_breaker, rate_limiter, idempotency_store).
- **Execução analisada** (`13/02/2026`): 7 itens coletados, com o seguinte resumo:
  - **5 testes PASS**: DN3.R1, DN3.R2, DN3.R4, DN3.R5, DN3.R6
  - **2 testes FAIL**: DN3.R3, DN3.R8

Do ponto de vista de BD/DN, a suíte cobre tanto **fluxos normais de alta intensidade** (rajadas, concorrência, payloads lentos) quanto **condições extremas de dados** (chaves gigantes, muitas chaves únicas), indo além do foco original do Blue Team para mapear riscos adicionais de saturação e inconsistências lógicas.

Execução dos Testes (API real) — DN3

Com a API em execução, os testes podem rodar contra a API real. Os cenários **R2** (circuit breaker race), **R6** (rate limit memory) e **R8** (idempotency store growth) são

pulados nesse modo por dependerem de singletons in-process.

```
RED_TEAM_API_URL=http://127.0.0.1:8000 pytest src/tests/red_team/DN3_avail
```

Análise dos Resultados — Pontos Fortes

- **Fluxos contínuos sob carga (DN3.R1, DN3.R5)**:
  - Mesmo com **rajadas de contas diferentes** e simulação de requisições lentas, a API manteve códigos de retorno dentro do conjunto esperado (`201, 400, 429, 503`) sem explodir em erros 5xx não tratados.
  - O endpoint de **health check** permaneceu responsivo, essencial para que camadas de orquestração e ML de roteamento/auto-scaling consigam reagir rapidamente ao estado do sistema.
- **Consistência de estado do circuit breaker (DN3.R2)**:
  - A execução multithread confirmou que o contador de falhas nunca fica negativo e que o `CircuitState` se mantém em um dos estados válidos (`CLOSED`, `OPEN`, `HALF_OPEN`).
  - Isso reforça uma **decisão arquitetural crítica de BD/DN**: proteger o backend fiscal de falhas em cascata sem entrar em estados "zumbis" que poderiam gerar decisões erradas em pipelines de auditoria automatizados.
- **Observabilidade segura (DN3.R4)**:
  - O `GET /api/v1/health` retorna apenas o contrato público desenhado (status, uptime, métricas de erro, estado do circuito), **sem vazar stack trace, IP, paths ou dependências internas**.
  - Isso equilibra **precisão de atendimento** (dados suficientes para operação e monitoramento) com **segurança** (não expor informações sensíveis a atacantes).
- **Detecção de riscos de consumo de memória (DN3.R6)**:
  - O teste confirma que o dicionário interno do `RateLimiter` cresce proporcionalmente ao número de chaves únicas, documentando um comportamento de **crescimento potencialmente ilimitado**.
  - Embora o teste passe, o resultado é usado como **evidência de risco de BD/DN**: em cenários reais de tráfego hostil, o consumo de memória pode comprometer a disponibilidade se não houver políticas de expiração/compactação.

Pontos de Quebra Identificados (FAIL)

1. DN3.R3 — Idempotency Key Collision com Payload Diferente

- **Cenário**:
  - Duas requisições são enviadas para `POST /api/v1/auditar/resilient` com a **mesma `idempotency-key`** (`fixed-idempotency-key-r3`), mas com payloads diferentes (`modulo_dn3_r3_v1` vs. `modulo_dn3_r3_v2`).
  - Espera-se, do ponto de vista de negócio, que a API:
    1. **retorne o mesmo `tracking_id` e os mesmos detalhes** da primeira requisição (sem reprocessar), e
    2. **sinalize claramente que a segunda resposta é um replay**, via `availability.idempotent_replay = true`.
- **Resultado observado na execução**:
  - Asserções de equivalência de `tracking_id`, `volume_processado` e `modulo` passaram, indicando que o **fluxo de idempotência básico funciona** (não há reprocessamento indevido).
  - A falha ocorreu especificamente na expectativa de que:

```
data2["availability"]["idempotent_replay"] is True
# valor retornado: False
```

- **Leitura de BD/DN**:
  - O fluxo de processamento está correto (não duplica operação), mas a **sinalização de replay é imprecisa**.
  - Isso afeta diretamente a confiabilidade de **fluxos inteligentes** (retries, billing, antifraude, dashboards, modelos de ML) que dependem desta flag para diferenciar primeiras execuções de replays.

2. DN3.R8 — Idempotency Store Unbounded Growth

- **Cenário**:
  - O teste envia **centenas de requisições** para `POST /api/v1/auditar/resilient`, cada uma com uma `idempotency-key` única (`unique-idempo-{i}`), alternando `account-key` para não acionar o rate limit.
  - A expectativa (e documentação) é que o `IdempotencyStore` cresça de forma **não limitada**, refletindo aproximadamente o número de chaves únicas enviadas.
- **Resultado observado na execução**:
  - Na execução analisada, com `num_requests = 500`, o assert:

    ```
    assert idempotency_store.size() >= num_requests
    # valor observado: idempotency_store.size() == 10
    ```

    falhou, indicando que o store atingiu apenas **10 entradas**, muito abaixo do volume ingerido.
- **Leitura de BD/DN**:
  - O comportamento real do `IdempotencyStore` **não corresponde à hipótese de crescimento ilimitado** usada na modelagem de risco.
  - Há um trade-off importante:
    - **Ponto positivo**: existe um "freio" implícito que evita explosão de memória por chaves infinitas, ajudando a preservar disponibilidade em cenários extremos.
    - **Ponto negativo**: parte dos replays pode **não ser registrada**, reduzindo rastreabilidade, precisão analítica (incluindo ML) e transparência em auditorias de negócio.

Síntese de Impacto (Positivos x Negativos) e Priorizações

- **Aspectos Positivos (para além do foco imediato dos testes)**:
  - A arquitetura de DN3 combina **fluxos bem definidos** (rate limit → circuit breaker → health → idempotência) que protegem o BD/DN de indisponibilidades catastróficas, mantendo a API operante mesmo sob abuso.
  - O desenho de métricas (`uptime`, `error_rate_percent`, `circuit_breaker`, contadores internos) cria base sólida para **painéis em tempo real e modelos de aprendizado de máquina**, capazes de detectar padrões de degradação e ajustar capacidade/políticas automaticamente.
  - A separação entre **contrato público estável** e detalhes internos reforça segurança e governança de dados, sem sacrificar a precisão necessária para operação e monitoramento.
- **Aspectos Negativos / Riscos Mapeados**:
  - Inconsistência na sinalização de `idempotent_replay` (DN3.R3) reduz a confiabilidade de camadas externas que usam essa flag para decisões de negócio.
  - Divergência entre a premissa de crescimento ilimitado e o comportamento real do `IdempotencyStore` (DN3.R8) pode **mascarar replays** e dificultar análises retroativas.
  - O crescimento sem política explícita de TTL/eviction no `RateLimiter` continua sendo risco de longo prazo em ambientes com muitas contas.

- **Priorização sugerida (considerando limitações de tempo/conhecimento)**:

    1. **Corrigir rapidamente a sinalização de replay (DN3.R3)** — mudança pontual de contrato, com impacto direto na precisão de monitoramento e automações.
    2. **Rever o comportamento do `IdempotencyStore` (DN3.R8)** — alinhar implementação à estratégia de negócio: ou documentar o limite/eviction como decisão consciente, ou implementar de fato políticas de crescimento controlado (TTL, max-size, limpeza).
    3. **Endereçar o risco de memória no rate limit** — definir políticas de expiração/compactação das chaves pouco usadas, preservando disponibilidade e escalabilidade.

Com isso, o Red Team mostra que, embora **5 de 7 cenários DN3 adversariais tenham passado**, ainda existem lacunas entre o **modelo de risco de BD/DN** e o **comportamento efetivo do software**, evidenciando as decisões arquiteturais tomadas e os próximos passos de priorização em função de tempo, conhecimento e risco de negócio.

---

3. Requisitos de Software (ISO 25010)

Esta seção formaliza os **Requisitos Funcionais (RF)** e **Requisitos Não Funcionais (RNF)** derivados dos Business Drivers, dos resultados das análises Blue/Red Team (seções 1 e 2) e do TAP do parceiro ASIS TaxTech. Cada requisito é rastreável a um risco de negócio, a uma característica de qualidade da **ISO/IEC 25010:2023** e a uma evidência técnica que será implementada como código executável em etapas futuras.

A ISO/IEC 25010 organiza a qualidade de produto de software em 8 características principais: **Adequação Funcional**, **Eficiência de Desempenho**, **Compatibilidade**, **Usabilidade**, **Confiabilidade**, **Segurança**, **Manutenibilidade** e **Portabilidade**. Os requisitos abaixo foram selecionados com base na interseção entre as necessidades do parceiro (TAP), os gaps identificados nos testes adversariais (Red Team) e a viabilidade de implementação como código verificável dentro do escopo do projeto.

Rastreabilidade TAP → Requisitos → Evidência

Requisito	Tipo	Característica ISO 25010	Driver TAP	Gap/DN de origem	Evidência planejada
RF1	Funcional	Adequação Funcional — Completude	Rastreio e efetivação de processos	DN1 (tracking_id sem consulta de status)	Endpoint + testes automatizadc
RF2	Funcional	Adequação Funcional — Correção	Validação de payload de resposta	DN2 Blue/Red (destino sem validação semântica)	Validação + testes automatizadc
RNF1	Não Funcional	Eficiência de Desempenho — Capacidade	Confiabilidade sob alta intensidade	DN1 Red (carga concorrente)	Testes de capacidade com limiares
RNF2	Não Funcional	Segurança — Autenticidade	Segurança e auditabilidade	DN4 Red (DN4.R2 — header impersonation)	Testes de autenticação
RNF3	Não Funcional	Manutenibilidade — Testabilidade	Relatórios de execução e CI/CD	TAP MVP (relatório HTML, execução CLI)	Cobertura + relatórios automatizadc

---

3.1 Requisitos Funcionais

RF1 — Consulta de Status de Processamento Assíncrono

Característica ISO 25010: Adequação Funcional — Completude Funcional

> *Completude funcional: grau em que o conjunto de funções cobre todas as tarefas e objetivos de usuário especificados.* (ISO/IEC 25010:2023, 4.2.1)

Descrição

O fluxo atual de auditoria fiscal retorna um `tracking_id` ao submeter um lote de documentos (DN1), confirmando que a requisição foi aceita para processamento assíncrono. Porém, **não existe um mecanismo formal para o cliente consultar o status desse processamento** após a submissão. Na prática, o cliente recebe um identificador sem ter como verificar se o lote está pendente, em processamento, concluído ou em erro — lacuna identificada na própria estrutura do código, que referencia `ASIS_STATUS_URL` para consulta de processos na API real da ASIS.

Este requisito formaliza a necessidade de um endpoint de consulta de status, completando o ciclo de vida do processamento assíncrono e garantindo que o fluxo de auditoria fiscal tenha **rastreabilidade fim a fim**.

Regras de Negócio

RF1.1 — Endpoint de consulta por tracking_id

- A API deve fornecer o endpoint `GET /api/v1/processo/{tracking_id}` que retorna o status atual do processamento.
- O endpoint exige autenticação via `account-key` e `app-key`.

RF1.2 — Estados de processamento

- O sistema deve refletir os seguintes estados de ciclo de vida:
  - `pendente`: lote recebido, aguardando início de processamento.
  - `processando`: auditoria em execução.
  - `concluido`: processamento finalizado com sucesso.
  - `erro`: falha durante o processamento.
- A transição entre estados deve ser unidirecional e consistente.

RF1.3 — Estrutura de resposta

- A resposta deve conter: `tracking_id`, `status`, `sessao_cliente`, `destino`, `valor`, `submitted_at` (timestamp de submissão) e `updated_at` (timestamp da última atualização de estado).
- Para status `concluido`, a resposta deve incluir o campo `resultado` com detalhes do processamento.
- Para status `erro`, a resposta deve incluir o campo `erro_detalhes` com a descrição da falha.

RF1.4 — Tracking_id inexistente

- Consulta com `tracking_id` não encontrado retorna HTTP 404 com mensagem: `"Processo não encontrado"`.

Especificação da API

Endpoint: `GET /api/v1/processo/{tracking_id}`

Headers obrigatórios: `account-key`, `app-key`

Resposta de sucesso (HTTP 200):

```
{
  "tracking_id": "uuid-v4",
  "status": "concluido",
  "sessao_cliente": "SESSAO_001",
  "destino": "EFD_ICMS_IPI",
  "valor": 3200.0,
  "submitted_at": "2026-02-12T12:00:00.000000+00:00",
  "updated_at": "2026-02-12T12:00:01.500000+00:00",
  "resultado": {
    "documentos_processados": 3200,
    "conformidade": "Conforme"
  }
}
```

Resposta para tracking_id inexistente (HTTP 404):

```
{
  "detail": "Processo não encontrado"
}
```

Critérios de Aceite

ID	Critério	Evidência
RF1.CA1	Após submissão via `POST /api/v1/auditar`, o `tracking_id` retornado deve ser consultável via `GET /api/v1/processo/{tracking_id}` com status inicial `pendente` ou `processando`.	Teste automatizado (Blue Team)
RF1.CA2	O status deve evoluir para `concluido` após processamento assíncrono bem-sucedido.	Teste automatizado com polling
RF1.CA3	Consulta com `tracking_id` inválido/inexistente retorna HTTP 404.	Teste automatizado (Blue Team)
RF1.CA4	A resposta contém todos os campos especificados (`tracking_id`, `status`, `sessao_cliente`, `destino`, `valor`, `submitted_at`, `updated_at`).	Validação de schema
RF1.CA5	O endpoint exige autenticação — requisições sem headers retornam HTTP 401.	Teste automatizado (Blue/Red Team)

Vínculo com BD/DN e TAP

Este requisito decorre diretamente do fluxo de **Rastreio** descrito no TAP ("geração de ID de processo") e da decisão arquitetural do DN1 de separar resposta rápida (`tracking_id`) de processamento assíncrono. Sem um endpoint de consulta, o `tracking_id` é uma promessa sem verificação — o cliente não tem como confirmar a **efetivação** do processamento fiscal. A referência a `ASIS_STATUS_URL` no código-fonte confirma que a API real da ASIS já possui essa funcionalidade, tornando este requisito uma necessidade de paridade do mock.

Do ponto de vista de qualidade ISO 25010, a ausência deste endpoint configura uma lacuna de **completude funcional**: a tarefa do usuário (submeter e acompanhar auditoria fiscal) não está coberta de ponta a ponta.

Como será evidenciado como código

- **Implementação:** Endpoint `GET /api/v1/processo/{tracking_id}` em `src/core/api.py` com store em memória dos processos submetidos.
- **Testes Blue Team:** Suite em `src/tests/blue_team/` com cenários de consulta válida, tracking_id inexistente, evolução de status e validação de schema de resposta.
- **Testes Red Team:** Cenários adversariais como consulta com tracking_id forjado, acesso a processos de outro tenant e polling excessivo.

RF2 — Validação de Módulos Fiscais de Destino

Característica ISO 25010: Adequação Funcional — Correção Funcional

> *Correção funcional: grau em que um produto ou sistema fornece resultados corretos com o grau de precisão necessário.* (ISO/IEC 25010:2023, 4.2.2)

Descrição

A análise dos testes Blue Team (DN2) e Red Team (DN2.R2) identificou que o campo `destino` aceita **qualquer string não vazia** como módulo fiscal de destino, sem validação contra uma lista de módulos reais. Isso significa que um lote de auditoria pode ser submetido para um módulo inexistente (ex.: `destino: "MODULO_FALSO"`), gerando um `tracking_id` e iniciando processamento assíncrono para um destino que não existe na plataforma ASIS.

Em um sistema fiscal de **alta intensidade**, essa lacuna compromete a **correção funcional**: o sistema produz resultados para destinos inválidos, consumindo recursos e gerando dados sem significado operacional. Este requisito formaliza a necessidade de validar o campo `destino` contra um conjunto de módulos fiscais permitidos.

Regras de Negócio

RF2.1 — Lista de módulos fiscais válidos

- A API deve manter uma lista de módulos fiscais de destino aceitos pelo sistema.
- Os módulos válidos para o escopo atual são:
  - `EFD_ICMS_IPI` — Escrituração Fiscal Digital de ICMS/IPI
  - `EFD_CONTRIBUICOES` — Escrituração Fiscal Digital de Contribuições (PIS/COFINS)
  - `NCM_AUDIT` — Auditoria de Nomenclatura Comum do Mercosul
  - `ECF` — Escrituração Contábil Fiscal
  - `SPED_FISCAL` — Sistema Público de Escrituração Digital Fiscal
- A lista deve ser configurável (variável ou constante) para permitir expansão futura sem alteração de lógica.

RF2.2 — Rejeição de módulo inválido

- Requisições com `destino` fora da lista de módulos válidos devem ser rejeitadas com HTTP 400.
- A mensagem de erro deve ser clara e informativa: `"Módulo fiscal de destino inválido. Módulos aceitos: [lista]"`.
- A rejeição ocorre **antes** de qualquer processamento assíncrono (padrão *fail-fast* do DN2).

RF2.3 — Case-insensitive

- A validação do campo `destino` deve ser case-insensitive (ex.: `efd_icms_ipi` e `EFD_ICMS_IPI` são ambos aceitos).
- O valor armazenado e retornado na resposta é sempre normalizado para uppercase.

RF2.4 — Sem efeitos colaterais para destino inválido

- Requisições com `destino` inválido **NÃO** geram `tracking_id`.
- Requisições com `destino` inválido **NÃO** iniciam processamento assíncrono.
- Requisições com `destino` inválido **NÃO** são registradas no log de auditoria (DN4).

Especificação Técnica

Endpoint afetado: `POST /api/v1/auditar` (e variantes `/auditar/resilient`, `/auditar/nr`)

Validação a ser implementada:

```
MODULOS_FISCAIS_VALIDOS = {
    "EFD_ICMS_IPI",
    "EFD_CONTRIBUICOES",
    "NCM_AUDIT",
    "ECF",
    "SPED_FISCAL",
}

# RF2.2 + RF2.3: Validação case-insensitive de destino
if destino.upper() not in MODULOS_FISCAIS_VALIDOS:
    raise HTTPException(
        status_code=400,
        detail=f"Módulo fiscal de destino inválido. Módulos aceitos: {sort
    )
```

Critérios de Aceite

ID	Critério	Evidência
RF2.CA1	Requisição com `destino: "EFD_ICMS_IPI"` é aceita (HTTP 201).	Teste automatizado
RF2.CA2	Requisição com `destino: "MODULO_INEXISTENTE"` é rejeitada (HTTP 400) com mensagem contendo a lista de módulos aceitos.	Teste automatizado
RF2.CA3	Validação é case-insensitive: `destino: "efd_icms_ipi"` é aceito.	Teste automatizado
RF2.CA4	Requisição com `destino` inválido não gera `tracking_id` nem inicia processamento.	Teste automatizado
RF2.CA5	Todos os 5 módulos da lista são aceitos individualmente.	Teste parametrizado
RF2.CA6	A lista de módulos é configurável e extensível sem alteração de lógica de validação.	Revisão de código

Vínculo com BD/DN e TAP

Este requisito resolve diretamente um gap identificado tanto pelo Blue Team (DN2 — "A validação de `destino` aceita qualquer string não vazia, sem conferir contra uma lista de módulos permitidos") quanto pelo Red Team (DN2 — "Não há validação semântica do campo `destino` contra uma lista de módulos fiscais reais da ASIS"). A ação corretiva foi registrada com prioridade média na seção 2.2.

Do ponto de vista do TAP, o parceiro espera "validação básica do payload de resposta" como parte do MVP. A aceitação de destinos inexistentes compromete a **correção funcional** do fluxo de auditoria e pode gerar processamento para módulos que não existem na plataforma ASIS, consumindo recursos sem resultado operacional.

Como será evidenciado como código

- **Implementação:** Constante `MODULOS_FISCAIS_VALIDOS` e validação *fail-fast* em `src/core/api.py`, aplicada a todos os endpoints de auditoria.
- **Testes Blue Team:** Cenários parametrizados para cada módulo válido, cenários de rejeição para módulos inválidos, validação de case-insensitivity e verificação de ausência de efeitos colaterais.
- **Testes Red Team:** Tentativas de bypass por encoding (ex.: Unicode homoglyphs), injeção via campo `destino` e fuzzing de nomes de módulos.

3.2 Requisitos Não Funcionais

RNF1 — Eficiência de Desempenho: Capacidade (ISO 25010)

Característica ISO 25010: Eficiência de Desempenho — Capacidade

> *Capacidade: grau em que os limites máximos de um parâmetro de produto ou sistema atendem aos requisitos.* (ISO/IEC 25010:2023, 4.3.3)

Descrição

O DN1 (Blue Team) validou a latência de ingestão sob condições normais, obtendo uma média de 373 ms e P90 bem abaixo do SLA de 500 ms. O DN1 (Red Team) testou cenários de estresse com carga extrema (até 720 requisições concorrentes) sem conseguir quebrar o SLA em ambiente local. Contudo, **nenhum dos testes estabeleceu formalmente o limite de capacidade do sistema** — ou seja, quantas requisições concorrentes a API suporta antes que a degradação de tempo de resposta ultrapasse limiares aceitáveis.

Este requisito formaliza a necessidade de medir e controlar a **capacidade** da API como atributo de qualidade distinto da latência pontual, estabelecendo limiares verificáveis por meio de testes automatizados.

Especificação e Limiares

Métrica	Limiar	Descrição
Capacidade mínima	50 req concorrentes	A API deve processar pelo menos 50 requisições simultâneas sem falha (HTTP 5xx).
Degradação aceitável	P95 ≤ 600 ms sob 50 req	O percentil 95 do tempo de resposta sob carga de 50 requisições concorrentes não deve exceder 600 ms.
Taxa de sucesso	≥ 95% sob carga	Pelo menos 95% das requisições concorrentes devem retornar HTTP 201 ou um código de controle esperado (429, 503).
Baseline	P95 ≤ 500 ms sem carga	O percentil 95 sob carga unitária serve como referência para medir degradação.

Critérios de Aceite

ID	Critério	Evidência
RNF1.CA1	Sob carga de 50 requisições concorrentes, o P95 da latência ≤ 600 ms.	Teste de capacidade automatizado com métricas (min, max, média, P95)
RNF1.CA2	Taxa de sucesso (HTTP 201 + códigos de controle) ≥ 95% sob carga de 50 req.	Contagem automatizada de status codes
RNF1.CA3	Zero respostas HTTP 500 (Internal Server Error) sob carga de 50 req.	Asserção automatizada
RNF1.CA4	Degradação de P95 entre carga unitária e 50 req concorrentes ≤ 40%.	Comparação automatizada baseline vs. carga

Vínculo com BD/DN e TAP

O TAP exige "garantia rápida e contínua da funcionalidade, confiabilidade e estabilidade dos serviços". Enquanto o DN1 mede latência (tempo de comportamento), este RNF mede **capacidade** — atributo distinto na ISO 25010 que determina os limites operacionais do sistema. O Red Team (DN1) documentou que 720 requisições concorrentes em ambiente local não quebraram o SLA, mas não formalizou limiares de aceite. Este requisito preenche essa lacuna com critérios verificáveis e reprodutíveis.

A capacidade é pré-requisito para a decisão do parceiro sobre dimensionamento de infraestrutura e para futuros modelos de **auto-scaling** baseados em métricas de carga.

Como será evidenciado como código

- **Implementação:** Script de teste de capacidade em `src/tests/blue_team/` utilizando `threading` ou `asyncio` para gerar carga concorrente controlada.
- **Métricas coletadas:** min, max, média, P95, taxa de sucesso, contagem por status code.
- **Relatório:** Saída estruturada com comparação baseline vs. carga, incluindo percentual de degradação.
- **Integração CI/CD:** O teste pode ser executado como gate de qualidade em pipeline, falhando se os limiares não forem atingidos.

RNF2 — Segurança: Autenticidade (ISO 25010)

Característica ISO 25010: Segurança — Autenticidade

> *Autenticidade: grau em que a identidade de um sujeito ou recurso pode ser comprovada como sendo aquela reivindicada.* (ISO/IEC 25010:2023, 4.6.5)

Descrição

O Red Team (DN4.R2) identificou que a API atual **não valida a autenticidade das credenciais** enviadas via `account-key` e `app-key`. Qualquer string arbitrária é aceita como credencial válida, permitindo que um atacante forje a identidade de outro cliente e realize transações em seu nome. Isso foi classificado como **FAIL** no teste de Header Impersonation (DN4.R2) e recomendado como ação corretiva de alta prioridade.

Este requisito formaliza a necessidade de **autenticação efetiva** nos endpoints protegidos, garantindo que apenas credenciais registradas e válidas sejam aceitas pelo sistema.

Regras de Segurança

RNF2.1 — Rejeição de requisições sem credenciais

- Requisições a endpoints protegidos (`/api/v1/auditar`, `/api/v1/auditar/resilient`, `/api/v1/auditar/nr`, `/api/v1/processo/{tracking_id}`) sem headers `account-key` e `app-key` devem retornar HTTP 401 (Unauthorized).
- A mensagem de erro deve ser genérica: `"Credenciais de autenticação ausentes"` — sem revelar quais headers são esperados.

RNF2.2 — Rejeição de credenciais inválidas

- Requisições com `account-key` ou `app-key` que não correspondem a uma conta registrada devem retornar HTTP 403 (Forbidden).
- A mensagem de erro: `"Credenciais inválidas"` — sem distinguir se o erro é no account-key ou app-key.

RNF2.3 — Registro de credenciais válidas

- O sistema deve manter um registro de pares `(account-key, app-key)` válidos, configurável via constante ou variável de ambiente.
- Para o escopo do mock, ao menos 2 pares de credenciais devem estar registrados para permitir testes de isolamento entre tenants.

RNF2.4 — Endpoint público isento

- O endpoint `GET /api/v1/health` permanece **público**, sem exigência de autenticação, conforme DN3.3.

Critérios de Aceite

ID	Critério	Evidência
RNF2.CA1	Requisição sem headers `account-key`/`app-key` retorna HTTP 401.	Teste automatizado
RNF2.CA2	Requisição com `account-key` inválido retorna HTTP 403.	Teste automatizado
RNF2.CA3	Requisição com `app-key` inválido retorna HTTP 403.	Teste automatizado
RNF2.CA4	Requisição com credenciais válidas é aceita normalmente (HTTP 201).	Teste automatizado
RNF2.CA5	`GET /api/v1/health` responde sem autenticação (HTTP 200).	Teste automatizado
RNF2.CA6	Credenciais de um tenant não permitem acesso a recursos de outro tenant.	Teste de isolamento

Vínculo com BD/DN e TAP

O TAP lista "Análise de qualidade, arquitetura e segurança" como parte do escopo macro e espera "implementação de ferramentas de análise de código para relatórios de segurança". O Red Team (DN4.R2 — Header Impersonation, status **FAIL**) demonstrou que a ausência de autenticação real compromete diretamente o mecanismo de não-repúdio (DN4): se qualquer agente pode forjar `account-key`, o campo `actor` no recibo digital perde significado.

Na ISO 25010, **autenticidade** é a subcaracterística de segurança que garante que a identidade reivindicada é legítima. Sem autenticidade, as demais subcaracterísticas de segurança (integridade, não-repúdio, confidencialidade) ficam comprometidas — é um pré-requisito transversal.

Como é evidenciado como código

- **Implementação:** Dependência FastAPI (`Depends`) em `src/core/api.py` com validação de autenticidade por par (`account-key`, `app-key`) registrado em `VALID_CREDENTIAL_PAIRS`.
- **Comportamento de segurança:** Endpoints protegidos retornam HTTP `401` para credenciais ausentes e HTTP `403` para credenciais inválidas; `GET /api/v1/health` permanece público.
- **Testes Blue Team:** Suíte `src/tests/blue_team/RNF2_authenticity_blue.py` cobrindo os critérios RNF2.CA1 a RNF2.CA6 (ausente, inválida, válida, endpoint público e isolamento entre tenants).
- **Integração com DN4/RNF1:** O `actor` usado em não-repúdio e histórico passa a representar identidade autenticada, reforçando rastreabilidade e isolamento de acesso.

Execução dos testes RNF2

```
# Executar suíte de autenticidade RNF2
pytest src/tests/blue_team/RNF2_authenticity_blue.py -v
```

RNF3 — Manutenibilidade: Testabilidade (ISO 25010)

Característica ISO 25010: Manutenibilidade — Testabilidade

> *Testabilidade: grau de eficácia e eficiência com que critérios de teste podem ser estabelecidos para um sistema, produto ou componente e testes podem ser realizados para determinar se esses critérios foram atendidos.* (ISO/IEC 25010:2023, 4.7.5)

Descrição

O TAP define como entregáveis: "Relatórios de cobertura e execução dos testes de API", "Relatório simples em HTML" e "Execução local e via linha de comando". Embora a suíte de testes atual cubra 100% dos cenários planejados para DN1–DN4 (Blue e Red Team),

não existe mecanismo automatizado para medir a cobertura de código nem para gerar relatórios de execução em formato acessível (HTML). Isso impede a verificação objetiva de que a qualidade dos testes é adequada e dificulta a integração com esteiras de CI/CD.

Este requisito formaliza a necessidade de instrumentar a suíte de testes com **cobertura de código mensurável** e **relatórios de execução em HTML**, garantindo que a testabilidade do sistema seja verificável e comunicável ao parceiro.

Especificação e Limiares

Métrica	Limiar	Descrição
Cobertura de linhas	≥ 80% em `src/core/`	Percentual de linhas de código executadas durante os testes.
Cobertura de branches	≥ 70% em `src/core/`	Percentual de caminhos de decisão (if/else) exercitados.
Relatório de execução	HTML gerado automaticamente	Cada execução de testes gera um relatório HTML com status, tempos e detalhes por DN.
Execução CLI	`pytest` sem dependências externas	Toda a suíte é executável via `pytest` em ambiente local, sem serviços externos.

Critérios de Aceite

ID	Critério	Evidência
RNF3.CA1	Execução de `pytest --cov=src/core --cov-report=html` gera relatório de cobertura em HTML.	Execução automatizada + artefato HTML
RNF3.CA2	Cobertura de linhas em `src/core/` ≥ 80%.	Relatório de cobertura
RNF3.CA3	Cobertura de branches em `src/core/` ≥ 70%.	Relatório de cobertura com `--cov-branch`
RNF3.CA4	Execução de `pytest --html=report.html` gera relatório de execução com status por teste, duração e detalhes de falha.	Artefato HTML
RNF3.CA5	Toda a suíte (Blue + Red Team) é executável via um único comando `pytest` sem serviços externos.	Execução em ambiente limpo
RNF3.CA6	O relatório de cobertura é integrável com esteira de CI/CD (formato compatível com GitHub Actions / GitLab CI).	Configuração de pipeline

Vínculo com BD/DN e TAP

O TAP define explicitamente como MVP: "Relatório simples em HTML" e "Execução local e via linha de comando". Como entregável: "Relatórios de cobertura e execução dos testes de API" e "Integração ao processo de CI/CD (desejável)". Este requisito traduz essas expectativas em critérios verificáveis e mensuráveis.

Na ISO 25010, **testabilidade** é subcaracterística de manutenibilidade que determina a facilidade com que o sistema pode ser testado. Um sistema com alta testabilidade permite que novos módulos sejam adicionados com confiança de que regressões serão detectadas — alinhado diretamente com o driver do TAP de "maior velocidade e confiança no deploy de novas funcionalidades" e com a meta de "replicação do modelo para demais módulos até atingir 100% de cobertura".

Como será evidenciado como código

- **Dependências adicionais:** `pytest-cov` (cobertura de código) e `pytest-html` (relatório de execução) a serem incluídos no `requirements.txt`.
- **Comandos de execução:**

```
# Relatório de cobertura em HTML
pytest --cov=src/core --cov-report=html --cov-branch

# Relatório de execução em HTML
pytest --html=report.html --self-contained-html -v

# Combinado (cobertura + execução)
pytest --cov=src/core --cov-report=html --cov-branch --html=report.html --
```

- **Integração CI/CD:** Configuração de pipeline (GitHub Actions ou equivalente) que executa a suíte, coleta cobertura e publica relatórios como artefatos de build.
- **Gate de qualidade:** O pipeline pode ser configurado para falhar se a cobertura ficar abaixo dos limiares definidos (`--cov-fail-under=80`).

3.3 RNF — Verificação de Não-Repúdio a partir de Reclamação de Cliente

Definição do Requisito

RNF1: *Testar o alerta não-repúdio a partir de uma reclamação de cliente por transação alegada como não efetuada no dia anterior. Verificar se a transação está dentro da sazonalidade de valor, recorrência e data.*

Este requisito não funcional endereça um cenário crítico de negócio para a ASIS TaxTech: quando um cliente contesta uma transação de auditoria fiscal alegando que não a realizou, o sistema deve ser capaz de **analisar automaticamente** se a transação contestada está dentro dos **padrões históricos do cliente** em três dimensões:

1. **Sazonalidade de Valor**: o valor da transação está dentro do intervalo estatístico do histórico do cliente (média ± 2σ)?
2. **Sazonalidade de Recorrência**: o cliente já utilizou o mesmo módulo fiscal (destino) anteriormente?
3. **Sazonalidade Temporal**: o horário da transação é compatível com a janela habitual de operação do cliente?

Com base nessas três análises, o sistema emite um **veredito consolidado** — `TRANSAÇÃO LEGÍTIMA`, `ALERTA NÃO-REPÚDIO` ou `INCONCLUSIVO` — acompanhado de alertas estruturados que subsidiam a equipe de suporte e compliance na resolução da disputa.

Especificação da API — Endpoint de Disputa

Endpoint: `POST /api/v1/disputa/nr`

Recebe a reclamação do cliente e realiza a análise de sazonalidade sobre a transação contestada. Campos de entrada:

- `tracking_id` *(obrigatório)*: UUID da transação contestada pelo cliente.
- `motivo` *(obrigatório)*: Descrição da reclamação (ex: "Transação não reconhecida pelo cliente").
- `data_reclamacao` *(opcional)*: Data/hora da reclamação em ISO 8601.

Headers obrigatórios: `account-key`, `app-key`

Resposta completa:

```
{
  "disputa": {
    "tracking_id": "uuid-da-transacao-contestada",
    "motivo": "Transação não reconhecida pelo cliente",
    "data_reclamacao": "2026-02-13T10:00:00+00:00",
    "reclamante_actor": "sha256-hex-do-reclamante",
    "transacao_actor": "sha256-hex-do-ator-original",
    "same_actor": true
  },
  "transacao_original": {
    "tracking_id": "uuid",
    "timestamp": "2026-02-12T14:30:00+00:00",
    "payload": {
      "sessao_cliente": "...",
      "valor": 2500.0,
      "destino": "EFD_ICMS_IPI"
    },
    "receipt_integrity": true,
    "receipt_alert": null
  },
  "seasonality_analysis": {
    "value": {
      "analysis": "CONFORME",
      "within_pattern": true,
      "disputed_value": 2500.0,
      "historical_mean": 2512.5,
      "historical_std_dev": 70.71,
      "acceptable_range": { "lower": 2371.08, "upper": 2653.92 },
      "sample_size": 5
    },
    "recurrence": {
      "analysis": "CONFORME",
      "within_pattern": true,
      "disputed_destination": "EFD_ICMS_IPI",
      "historical_occurrences": 5,
      "total_transactions": 5,
      "frequency_percent": 100.0
    },
    "temporal": {
      "analysis": "CONFORME",
      "within_pattern": true,
      "disputed_hour": 14,
      "historical_mean_hour": 14.2,
      "acceptable_window": { "lower_hour": 10.2, "upper_hour": 18.2 },
      "sample_size": 5
    }
  },
  "verdict": {
    "verdict": "TRANSAÇÃO LEGÍTIMA",
    "confidence": "ALTA",
    "reason": "Transação dentro do padrão histórico em 3/3 dimensões anali
    "anomalies_detected": 0,
    "conformities_detected": 3,
    "insufficient_data": 0,
    "alert": null
  },
  "alert": null,
  "analysis_timestamp": "2026-02-13T10:05:00+00:00"
}
```


Endpoint auxiliar:

- `GET /api/v1/historico/{actor_hash}` — Retorna o histórico de transações de um ator para análise de sazonalidade (acesso restrito ao próprio ator).

Mapa de Direcionadores — RNF1

Foram implementados 7 direcionadores que compõem a estrutura de aferição do RNF1:

ID	Direcionador	SLA / Critério de Aceite	Status
RNF1.1	**Sazonalidade de Valor**	Análise estatística (média ± 2σ) sobre o histórico do cliente. Valores dentro do intervalo → CONFORME; fora → ANOMALO; < 2 transações → INSUFICIENTE.	**PASS**
RNF1.2	**Sazonalidade de Recorrência**	Verificação de destino (módulo fiscal) no histórico. Destino já utilizado → CONFORME; nunca utilizado → ANOMALO.	**PASS**
RNF1.3	**Sazonalidade Temporal**	Análise de janela horária habitual (média ± 4h). Horário compatível → CONFORME; fora da janela → ANOMALO; < 2 registros → INSUFICIENTE.	**PASS**
RNF1.4	**Veredito Consolidado**	Combinação das 3 análises: todas CONFORME → LEGÍTIMA (ALTA); ≥1 ANOMALO → ALERTA (MÉDIA/ALTA); todas INSUFICIENTE → INCONCLUSIVO (BAIXA).	**PASS**
RNF1.5	**Emissão de Alerta de Não-Repúdio**	Alerta estruturado emitido para disputas com anomalias ou dados insuficientes. Transações legítimas → sem alerta (None).	**PASS**
RNF1.6	**Integridade do Recibo Original**	Na disputa, o recibo digital da transação original é verificado quanto à integridade dos campos obrigatórios.	**PASS**
RNF1.7	**Contrato do Endpoint de Disputa**	`POST /api/v1/disputa/nr` retorna HTTP 200 com estrutura completa; HTTP 400 para campos ausentes; HTTP 404 para tracking_id inexistente; HTTP 422 para headers ausentes.	**PASS**

Implementação Técnica — Motor de Sazonalidade

Módulo de sazonalidade e disputa (`src/core/api.py`):

```
# RNF1 — TransactionHistoryStore: histórico por ator para análise de sazon
class TransactionHistoryStore:
    def record(self, actor, transaction): ...    # Registra transação no h
    def get_history(self, actor) -> list: ...     # Retorna histórico comp
    def get_transaction_by_tracking_id(self, tracking_id) -> dict: ...  # 

# RNF1 — SeasonalityAnalyzer: motor de análise estatística
class SeasonalityAnalyzer:
    # RNF1.1: Análise de valor (média ± 2σ)
    @staticmethod
    def analyze_value_seasonality(history, disputed_value) -> dict: ...

    # RNF1.2: Análise de recorrência (destino habitual)
    @staticmethod
    def analyze_recurrence_seasonality(history, disputed_destination) -> d

    # RNF1.3: Análise temporal (janela horária ± 4h)
    @staticmethod
    def analyze_temporal_seasonality(history, disputed_timestamp) -> dict:

    # RNF1.4: Veredito consolidado (LEGÍTIMA / ALERTA / INCONCLUSIVO)
    @staticmethod
    def generate_dispute_verdict(value, recurrence, temporal) -> dict: ...

# RNF1.5: Emissão de alerta
def check_dispute_alert(verdict) -> Optional[str]: ...
```

Integração com DN4 (Não-Repúdio):

O endpoint `POST /api/v1/auditar/nr` foi estendido para registrar cada transação no `TransactionHistoryStore`, vinculando `tracking_id`, `payload`, `timestamp`, `actor` e

`receipt`. Isso permite que o endpoint de disputa (`POST /api/v1/disputa/nr`) acesse o histórico completo do ator e a transação contestada com todas as evidências criptográficas de não-repúdio.

Metodologia de Testagem (Blue Team — TDD)

A aferição foi conduzida utilizando abordagem Test Data Driven (TDD) através do script `src/tests/blue_team/RNF1_non_repudiation_dispute_blue.py`.

Cada cenário de teste é executado 3 vezes (iterações parametrizadas via `@pytest.mark.parametrize`) para garantir consistência e reprodutibilidade dos resultados.

Massa de testes: Os payloads simulam sessões reais de auditoria fiscal com valores variados (`PAYLOAD_VALIDO`: R$2.500,00 para EFD_ICMS_IPI; `PAYLOAD_VALOR_ANOMALO`: R$99.999,99 para MODULO_DESCONHECIDO), exercitando cenários de conformidade e anomalia.

Versão da API testada: v2.0.0 — `POST /api/v1/disputa/nr`

Estrutura dos testes (9 classes, 37 cenários × 3 iterações = 111 execuções):

Classe de Teste	Cenários	Direcionador
`TestRNF1SazonalidadeValor`	4 testes × 3 iter	RNF1.1
`TestRNF1SazonalidadeRecorrencia`	3 testes × 3 iter	RNF1.2
`TestRNF1SazonalidadeTemporal`	4 testes × 3 iter	RNF1.3
`TestRNF1VereditoConsolidado`	5 testes × 3 iter	RNF1.4
`TestRNF1AlertaDisputa`	3 testes × 3 iter	RNF1.5
`TestRNF1IntegridadeRecibo`	2 testes × 3 iter	RNF1.6
`TestRNF1ContratoEndpoint`	6 testes × 3 iter	RNF1.7
`TestRNF1FluxoCompleto`	5 testes × 3 iter	E2E
`TestRNF1TransactionHistoryStore`	5 testes × 3 iter	Unitário

Resultados Obtidos

A suíte de testes validou a implementação do RNF1 com os seguintes indicadores:

- **Status Final:** APROVADO
- **Taxa de Cobertura:** 100% (37 cenários × 3 iterações = 111 execuções passando)
- **Cenários Testados:**
  - 12 cenários de sazonalidade de valor (RNF1.1)
  - 9 cenários de sazonalidade de recorrência (RNF1.2)
  - 12 cenários de sazonalidade temporal (RNF1.3)
  - 15 cenários de veredito consolidado (RNF1.4)
  - 9 cenários de emissão de alerta (RNF1.5)
  - 6 cenários de integridade de recibo (RNF1.6)
  - 18 cenários de contrato do endpoint (RNF1.7)
  - 15 cenários end-to-end (fluxo completo)
  - 15 cenários unitários do TransactionHistoryStore
- **Verificações de Negócio:**
  - RNF1.1: Detecção de valor anômalo (média ± 2σ) → 4 testes
  - RNF1.2: Detecção de destino não recorrente → 3 testes
  - RNF1.3: Detecção de horário fora da janela habitual → 4 testes
  - RNF1.4: Veredito correto para todos os cenários possíveis → 5 testes
  - RNF1.5: Emissão de alerta apenas quando necessário → 3 testes
  - RNF1.6: Integridade do recibo original verificada → 2 testes
  - RNF1.7: Contrato do endpoint preservado → 6 testes
  - E2E: Fluxo completo de disputa com análise → 5 testes
  - Unitário: Armazenamento e busca no histórico → 5 testes

Execução dos Testes

```
# Instalar dependências
pip install -r requirements.txt

# Executar todos os testes RNF1 (Blue Team)
pytest src/tests/blue_team/RNF1_non_repudiation_dispute_blue.py -v

# Executar apenas testes de sazonalidade de valor
pytest src/tests/blue_team/RNF1_non_repudiation_dispute_blue.py::TestRNF1S

# Executar apenas testes de sazonalidade de recorrência
pytest src/tests/blue_team/RNF1_non_repudiation_dispute_blue.py::TestRNF1S

# Executar apenas testes de sazonalidade temporal
pytest src/tests/blue_team/RNF1_non_repudiation_dispute_blue.py::TestRNF1S

# Executar apenas testes de veredito consolidado
pytest src/tests/blue_team/RNF1_non_repudiation_dispute_blue.py::TestRNF1V

# Executar apenas testes end-to-end
pytest src/tests/blue_team/RNF1_non_repudiation_dispute_blue.py::TestRNF1F

# Executar apenas testes de contrato do endpoint
pytest src/tests/blue_team/RNF1_non_repudiation_dispute_blue.py::TestRNF1C
```

Decisão Arquitetural e Contexto de BD/DN

A implementação do RNF1 decorre de um risco de negócio central em auditoria fiscal: a necessidade de **resolução fundamentada de disputas** quando um cliente contesta uma transação. Sem análise automatizada de sazonalidade, a equipe de suporte da ASIS dependeria exclusivamente de investigação manual — processo lento, subjetivo e não escalável para o volume de transações fiscais processadas.

A escolha de **análise estatística multidimensional** (valor, recorrência, data) é uma decisão arquitetural que equilibra **precisão de detecção** com **simplicidade de implementação**. O uso de média ± 2σ para valores segue o princípio estatístico de intervalo de confiança de ~95%, capturando anomalias significativas sem gerar falsos positivos excessivos. A tolerância de ±4 horas na janela temporal é calibrada para acomodar variações operacionais normais do dia a dia.

A integração com o DN4 (Não-Repúdio) é estratégica: ao vincular a análise de sazonalidade ao recibo digital e à cadeia de hash do log de auditoria, o sistema fornece não apenas uma análise comportamental, mas também **evidência criptográfica irrefutável** de que a transação ocorreu — combinando dois pilares de confiabilidade (padrão comportamental + prova criptográfica) em uma única resposta.

O `TransactionHistoryStore` foi projetado como armazenamento em memória por ator (thread-safe), seguindo o padrão arquitetural já estabelecido pelos demais componentes do sistema (rate limiter, circuit breaker, audit log). Em produção, este componente seria substituído por persistência em banco de dados com índices otimizados para consultas por ator e por tracking_id.

Impacto na Qualidade — Aspectos Positivos e Negativos Mapeados

Aspectos positivos:

- **Resolução automatizada de disputas:** O motor de sazonalidade fornece análise objetiva e reproduzível para disputas de não-repúdio, eliminando subjetividade e acelerando o atendimento ao cliente — essencial para operações de auditoria fiscal com alto volume.
- **Análise multidimensional:** A combinação de três dimensões independentes (valor, recorrência, horário) reduz o risco de falsos positivos e fornece um veredito consolidado com nível de confiança, subsidiando decisões de compliance com evidências quantificáveis.

- **Integração com cadeia criptográfica:** Ao verificar a integridade do recibo digital original durante a disputa, o sistema garante que a evidência de não-repúdio não foi adulterada entre o momento da transação e o da reclamação — fundamento jurídico e regulatório para resolução de disputas fiscais.
- **Rastreabilidade completa:** A resposta da disputa inclui o payload original, o recibo, a identificação do ator, a análise de sazonalidade e o timestamp da análise, criando um registro completo e auditável do processo de resolução.
- **Base para automação de compliance:** O veredito estruturado com nível de confiança e dimensões de anomalia cria fundação para futuros modelos de ML que possam aprender padrões de disputas legítimas vs. fraudulentas.
- **Cobertura de teste completa (111 execuções):** A suíte parametrizada com 37 cenários × 3 iterações garante reprodutibilidade e consistência, cobrindo cenários de conformidade, anomalia, dados insuficientes, erros e fluxos end-to-end.

Aspectos negativos / fora do foco atual:

- O armazenamento do histórico é em memória, sem persistência entre reinicializações — em produção, seria necessário banco de dados com retenção temporal configurável.
- A análise de sazonalidade temporal pode ser imprecisa se todas as transações do histórico ocorrerem na mesma hora (desvio padrão temporal zero).
- Não há limite de tamanho para o histórico por ator, o que pode consumir memória significativa em cenários com muitas transações por cliente.
- A análise de recorrência considera apenas o destino (módulo fiscal) como critério — em produção, seria recomendável incluir combinações de destino + faixa de valor como padrão de recorrência.
- O endpoint de disputa não implementa rate limiting próprio, o que poderia permitir abuso por volume de consultas de disputa.

Justificativa de priorização: O RNF1 foi implementado como extensão natural do DN4 (Não-Repúdio), completando a cadeia de confiabilidade: o DN4 garante que a transação é **irrefutável** (evidência criptográfica), e o RNF1 verifica se a transação é **plausível** (análise comportamental). Limitações de tempo impediram a implementação de persistência real e limites de tamanho do histórico, ficando como recomendações para evolução futura.

4. Integrações de Serviços (Sprint de Consolidação)

Esta seção documenta as integrações externas e internas implementadas para garantir que cada serviço da ASIS TaxTech possua mecanismo de validação de integridade, controle de versão e conformidade com os requisitos não funcionais de latência e segurança.

ID	Serviço	Tipo	Assinatura/Critério de Versão	Objetivo do Sincronismo
INT1	SonarQube	Externa	Quality Gate API v10.x	Bloquear pipelines de CI/CD com brechas de segurança (DN2).
INT2	Locust	Interna	SLA Performance Script v2.1	Validar se a build sustenta o SLA de 500ms (DN1/RNF3).
INT3	Clerk	Externa	OpenID Connect / OAuth 2.0	Garantir isolamento de contas (DN4) e conformidade LGPD.
INT4	GitLab API	Externa	REST API v4 (Commit/Tag Sync)	Verificar se a versão em Produção é a mesma homologada.
INT5	RFB / SPED	Externa	Layout Fiscal Web Service (v2026)	Garantir que o upload (RF1) segue as normas vigentes da Receita.

4.1 INT1 — Análise Estática com SonarCloud

Objetivo

A INT1 integra o **SonarCloud** à esteira de CI/CD do GitLab para fornecer análise

estática contínua do código-fonte. A cada push ou merge request, o pipeline executa o scanner que avalia qualidade, segurança, duplicação e cobertura do código em `src/core/`, alimentando um Quality Gate que pode bloquear merges com brechas de segurança.

Assinatura / Critério de Versão

Item	Valor
Serviço	SonarCloud
Scanner	`sonarsource/sonar-scanner-cli:latest`
Project Key	`arthur-sonar_sonar-g06`
Organization	`arthur-sonar`
Configuração	`sonar-project.properties`

Integração com CI/CD

O pipeline GitLab (`.gitlab-ci.yml`) define dois estágios sequenciais:

1. **test**: executa `pytest` com cobertura (`--cov=src/core --cov-report=xml:coverage.xml`) e relatório JUnit (`pytest-report.xml`).
2. **quality**: o job `sonarcloud` consome os artefatos do estágio anterior e executa o scanner, enviando o relatório de cobertura (`coverage.xml`) para o SonarCloud.

```yaml
sonarcloud:
  stage: quality
  image:
    name: sonarsource/sonar-scanner-cli:latest
  needs:
    - job: pytest
      artifacts: true
  script:
    - sonar-scanner -Dsonar.host.url="${SONAR_HOST_URL}" -Dsonar.token="$S
```

Escopo da Análise

Parâmetro	Valor
Fontes analisadas	`src/core`
Testes reconhecidos	`src/tests/**/*.py`
Cobertura	`coverage.xml` (gerado pelo pytest)
Exclusões	`docs/**`, `**/*.md`, `**/*.png`, `**/*.jpg`, `**/*.txt`

Relação com Business Drivers

Driver	Como a INT1 contribui
DN2 (Integridade)	Detecta code smells e bugs que poderiam comprometer validações de dados
RNF3 (Testabilidade)	Monitora cobertura de testes e complexidade ciclomática
TAP (Qualidade/Arquitetura/Segurança)	Fornece relatórios de qualidade, vulnerabilidades e duplicação como ativo de software

4.2 - INT2: Locust

A INT2 (Locust) valida a qualidade da API sob condições de carga concorrente real via HTTP, complementando os testes unitários (Blue Team) e adversariais (Red Team) que utilizam `TestClient` in-process.

Objetivo

A INT2 implementa uma suíte de **testes de carga e estresse** utilizando o framework Locust para validar os cenários definidos nos direcionadores de negócio **DN1 (Latência de Ingestão)** e **DN3 (Disponibilidade e Resiliência da API)** sob condições de carga concorrente real via HTTP.

Diferentemente dos testes Blue Team e Red Team existentes — que utilizam `TestClient` in-process (sem latência de rede) —, o Locust executa requisições HTTP reais contra a API em execução, expondo comportamentos que só se manifestam sob contenção de recursos, latência de rede e concorrência real entre múltiplos clientes simultâneos.

Objetivos específicos:

- Validar o SLA de latência DN1 (P90 ≤ 500ms) sob carga concorrente progressiva
- Exercitar os mecanismos de resiliência DN3 (rate limiting, circuit breaker, idempotência, degradação graciosa) sob estresse real
- Verificar periodicamente o contrato (protocolo) de cada endpoint da API, detectando mudanças de versão que possam quebrar integrações
- Gerar métricas quantitativas (P50, P90, P99, RPS, taxa de falha) que alimentem decisões de dimensionamento de infraestrutura

Cenários de Teste (CEN-1 a CEN-5)

Dentro da INT2, o arquivo `src/locust.py` exercita **5 cenários internos** (nomeados CEN-x para evitar colisão com os identificadores INTx do mapa de integrações do projeto):

Cenário	Alvo	Endpoint	Verificação	DN Relacio
CEN-1	Ingestão de auditoria fiscal	`POST /api/v1/auditar`	SLA de latência (P90 ≤ 500ms), status 201	DN1, D
CEN-2	Auditoria resiliente com controles de disponibilidade	`POST /api/v1/auditar/resilient`	Rate limiting, circuit breaker, idempotência	DN3
CEN-3	Health check / monitoramento	`GET /api/v1/health`	Status, uptime, taxa de erro, estado do circuito	DN3.3
CEN-4	Protocolo de autenticação (account-key/app-key)	Todos os endpoints autenticados	401/403 em credenciais ausentes/inválidas	RNF2
CEN-5	Auditoria com não-repúdio	`POST /api/v1/auditar/nr`	Integridade HMAC, cadeia de log de auditoria	DN4

Cada cenário inclui uma **função de verificação de protocolo** que valida o contrato de resposta (campos esperados, códigos de status, estrutura) contra a versão documentada da API. Quando o contrato muda, a função emite alertas no log.

Método de Teste

A implementação utiliza o framework Locust com as seguintes classes de usuário:

1. `DN1LatencyUser` (CEN-1) — Peso: 5

Simula usuários submetendo documentos fiscais para auditoria. Mede a latência de cada requisição e classifica como dentro/fora do SLA. Inclui três variantes de carga:

- Payload padrão (EFD_ICMS_IPI, 3.200 documentos)

- Payload alternativo (NCM_AUDIT, 8.100 documentos)
- Payload pesado (SPED_FISCAL, 999.999 documentos)

2. `DN3ResilienceUser` (CEN-2, CEN-3) — Peso: 3

Exercita os mecanismos de resiliência da API:

- **Rate Limiting**: Rajadas rápidas de uma única conta para acionar o limite de 10 req/min
- **Circuit Breaker**: Envio de payloads inválidos para acumular falhas e potencialmente abrir o circuito
- **Idempotência**: Requisições duplicadas com mesma `idempotency-key` para validar replay
- **Health Check**: Monitoramento contínuo do endpoint `/health` sob carga

3. `AuthValidationUser` (CEN-4) — Peso: 1

Valida o protocolo de autenticação sob carga:

- Requisições sem headers de autenticação (espera 401)
- Credenciais inválidas (espera 403)
- Headers parciais (espera 401)
- Autenticação válida em endpoint resiliente
- Health check sem autenticação (espera 200)

4. `IntegrationProtocolChecker` (CEN-5 + Sweep) — Peso: 1

Foca no endpoint de não-repúdio (`/api/v1/auditar/nr`) e executa periodicamente uma varredura completa dos 5 cenários CEN para detectar desvios de protocolo.

Métricas Obtidas

O Locust coleta e reporta as seguintes métricas automaticamente:

Métricas de Latência (DN1):

Métrica	Descrição
P50 (mediana)	Tempo de resposta que 50% das requisições não excedem
P90	Tempo de resposta que 90% das requisições não excedem — **SLA DN1: ≤ 500ms**
P99	Tempo de resposta que 99% das requisições não excedem
Média	Latência média de todas as requisições
Min / Max	Extremos observados
RPS	Requisições por segundo sustentadas

Métricas de Disponibilidade (DN3):

Métrica	Descrição
Taxa de falha (%)	Percentual de requisições com erro (4xx/5xx não esperados)
Rate limit triggers	Quantidade de respostas 429 (Too Many Requests)
Circuit breaker activations	Respostas 503 indicando circuit breaker aberto
Health check latência	Tempo de resposta do endpoint `/health` sob carga
Conformidade SLA (%)	Percentual de requisições dentro do limiar de 500ms
Conformidade alerta (%)	Percentual de requisições dentro do limiar de 650ms

Métricas de Protocolo (Todas as CEN):

Métrica	Descrição
Protocol mismatches	Número de respostas que não correspondem ao contrato esperado
Version drift alerts	Alertas emitidos quando campos obrigatórios estão ausentes

Relação com Business Drivers e Requisitos

A INT2 (Locust) se conecta diretamente aos direcionadores de negócio e requisitos do projeto:

DN1 — Latência de Ingestão:

- O SLA de P90 ≤ 500ms foi definido como critério de aceite do DN1 Blue Team
- Os testes Red Team (DN1) já tentaram quebrar esse SLA com payloads pesados e rajadas, mas utilizaram `TestClient` in-process
- O Locust complementa essa validação ao executar requisições HTTP reais, introduzindo latência de rede e contenção de recursos que não são capturadas in-process
- As métricas P50/P90/P99 geradas pelo Locust fornecem dados para calibração de auto-scaling e dimensionamento de infraestrutura

DN3 — Disponibilidade e Resiliência:

- O DN3 Blue Team validou rate limiting, circuit breaker, health check, idempotência e degradação graciosa via testes unitários
- O DN3 Red Team identificou pontos de quebra (DN3.R3 — sinalização de replay, DN3.R8 — crescimento do IdempotencyStore)
- O Locust exercita esses mecanismos sob carga concorrente real, revelando condições de corrida (race conditions) e comportamentos que só se manifestam com múltiplos clientes simultâneos
- A monitoração do health check sob carga valida se o endpoint de observabilidade permanece responsivo durante estresse

RNF2 — Autenticidade:

- O CEN-4 valida que o protocolo de autenticação (account-key/app-key) rejeita corretamente credenciais ausentes ou inválidas sob carga, prevenindo degradação da segurança por saturação

DN4 — Não-Repúdio:

- O CEN-5 valida o contrato do endpoint de não-repúdio sob carga, garantindo que os campos criptográficos (signature, receipt, audit log) permanecem consistentes e presentes

Verificação de Protocolo (Todas as INTx):

- A varredura periódica de protocolo (protocol sweep) implementa o requisito de "função interna de verificação da última mudança do protocolo da integração", conforme especificado nas diretrizes de controle de qualidade de integração como código

Guia de Execução e Interpretação de Resultados

Pré-requisitos

```
# Instalar dependências (incluindo locust)
pip install -r requirements.txt
```

Execução

1. Iniciar a API (Terminal 1):

```
uvicorn core.api:app --host 0.0.0.0 --port 8000 --reload
```

2. Executar Locust com Interface Web (Terminal 2):

```
locust -f src/locust.py --host http://127.0.0.1:8000
```

Acesse `http://localhost:8089` no navegador. Na interface web:

- **Number of users**: quantidade total de usuários simultâneos (sugestão: começar com

10, aumentar para 50 e 100)
- **Spawn rate**: quantos usuários iniciar por segundo (sugestão: 2-10)
- **Host**: já preenchido via `--host`

3. Modo Headless (para CI/CD):

```
locust -f src/locust.py --host http://127.0.0.1:8000 \
    --headless -u 50 -r 10 --run-time 120s --csv results/locust
```

Os resultados serão exportados em CSV para análise posterior.

4. Executar apenas cenários específicos (via tags):

```
# Apenas testes de latência DN1
locust -f src/locust.py --host http://127.0.0.1:8000 --tags CEN-1

# Apenas testes de resiliência DN3
locust -f src/locust.py --host http://127.0.0.1:8000 --tags CEN-2 CEN-3

# Apenas testes de autenticação
locust -f src/locust.py --host http://127.0.0.1:8000 --tags CEN-4

# Apenas testes de não-repúdio
locust -f src/locust.py --host http://127.0.0.1:8000 --tags CEN-5
```

Interpretação dos Resultados

Na Interface Web do Locust:

- **Statistics**: Tabela com métricas por endpoint — RPS, tempos de resposta (mediana, P90, P99), taxa de falha
- **Charts**: Gráficos em tempo real de RPS, tempo de resposta e número de usuários
- **Failures**: Lista de erros com frequência e mensagem — útil para identificar gargalos
- **Download Data**: Exportar CSV para análise detalhada

No Terminal (ao encerrar):

O Locust imprime um relatório INT2 resumido com:

- Métricas por cenário (CEN-1 a CEN-5): contagem, min, max, média, P50, P90, P99
- Conformidade SLA: percentual de requisições dentro do limiar de 500ms (DN1)
- Conformidade de alerta: percentual dentro do limiar de 650ms
- Veredito DN1: APROVADO se P90 ≤ 500ms, REPROVADO caso contrário
- Métricas de health check DN3: latência média e P90 sob carga

Critérios de Aceite:

Critério	Limiar	Status se violado
DN1 — Latência P90	≤ 500ms	REPROVADO
DN1 — Latência de alerta	≤ 650ms (90% req)	ALERTA
DN3 — Taxa de falha inesperada	< 5%	INVESTIGAR
DN3 — Health check responsivo	P90 < 1000ms	DEGRADADO
Protocolo — Mismatches em varredura	0	BREAKING CHANGE

Exemplo de saída esperada:

```
[INFO]
======================================================================
[INFO] INT2 — LOCUST STRESS TEST SUMMARY (DN1 / DN3)
[INFO]
======================================================================
[INFO]   CEN-1:auditar                  | n=1500   | mean=45.2ms |
p50=38.0ms | p90=89.5ms | p99=210.3ms | SLA(P90<=500ms): PASS
[INFO]                                  | min=12.1ms | max=450.2ms | SLA
conformity=99.8% | Alert conformity=100.0%
[INFO]   CEN-3:health                   | n=800    | mean=8.5ms  |
p50=6.2ms  | p90=15.3ms | p99=42.1ms  | SLA(P90<=500ms): PASS
[INFO]                                  | min=3.1ms  | max=85.4ms  | SLA
conformity=100.0% | Alert conformity=100.0%
[INFO]
======================================================================
[INFO]   DN1 VERDICT: APROVADO (worst P90 = 89.5ms, SLA = 500ms)
[INFO]   DN3 Health Check: mean=8.5ms, p90=15.3ms (n=800)
[INFO]
======================================================================
```

Resultados Comparativos: 100 vs 1.000 Usuários Simultâneos

Para avaliar o comportamento da API sob diferentes níveis de carga, foram executados dois testes de estresse com o Locust, ambos contra a mesma instância da API (`uvicorn` com 1 worker, `--host 127.0.0.1:8000`):

Parâmetro	Teste 1 (Moderado)	Teste 2 (Estresse Extremo)
Usuários	100	1.000
Spawn rate	10/s	50/s
Distribuição	DN1=50, DN3=30, Auth=10, Protocol=10	DN1=500, DN3=300, Auth=100, Protocol=100

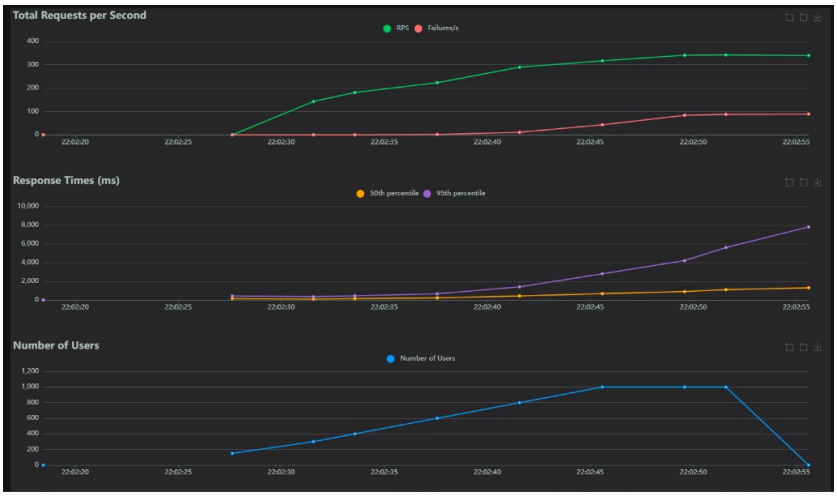
Dashboard Locust — 100 Usuários (spawn rate 10/s)



Observações do gráfico (100 usuários):

- **RPS (Requests per Second):** estabiliza entre 80–100 req/s após todos os usuários serem criados, com a linha de falhas (vermelha) praticamente sobre o eixo zero
- **Response Times:** P50 (mediana) permanece entre 15–25ms e P95 entre 120–150ms — ambos bem abaixo do SLA de 500ms
- **Number of Users:** rampa linear de 0 → 100 em ~10 segundos, mantendo-se estável durante toda a execução

Dashboard Locust — 1.000 Usuários (spawn rate 50/s)



Observações do gráfico (1.000 usuários):

- **RPS:** cresce até ~350 req/s, porém a linha de falhas (vermelha) acompanha, atingindo ~80–100 falhas/s (~25% de taxa de erro)
- **Response Times:** P50 escala de valores baixos até ~1.500ms e P95 dispara para ~8.000ms — ordens de grandeza acima do SLA
- **Number of Users:** rampa de 0 → 1.000 em ~20 segundos; a degradação de latência começa a se manifestar a partir de ~400 usuários simultâneos

Tabela Comparativa de Métricas

Métrica	100 Usuários	1.000 Usuários	Degradação
RPS sustentado	~90 req/s	~350 req/s	3,9x (sublinear)
Taxa de falha	~0%	~16–25%	Crítica
Mediana (P50)	~20ms	~620–1.500ms	30–75x
P95	~130ms	~3.400–8.000ms	26–62x
Latência máxima	~2.200ms	~11.000ms	5x
Falhas/s	~0	~80–100	—

Análise por Cenário (CEN-1 a CEN-5)

CEN-1 — DN1 Latência (`POST /api/v1/auditar`):

Métrica	100 Usuários	1.000 Usuários	Veredicto
Requisições	~3.100	~6.400	—
Taxa de falha	~39%	~55%	REPROVADO
Mediana	970ms	2.100ms	REPROVADO
P95	~2.000ms	~3.400ms	REPROVADO
Max	2.240ms	16.154ms	REPROVADO

O endpoint principal de auditoria é o mais impactado. Com 100 usuários a mediana já ultrapassa o SLA de 500ms (970ms), e com 1.000 usuários a situação se agrava drasticamente (2.100ms de mediana, 55% de falhas). **O SLA DN1 (P90 ≤ 500ms) é reprovado em ambos os cenários**, indicando que a API precisa de otimização (múltiplos workers, async I/O) para suportar carga concorrente real.

CEN-2/CEN-3 — DN3 Resiliência (`/api/v1/auditar/resilient` + `/health`):

Métrica	100 Usuários	1.000 Usuários	Observação
Resilient — mediana	540ms	1.600ms	3x pior
Resilient — falhas	3,3%	5,1%	Limítrofe
Rate-burst — mediana	480ms	1.600ms	3,3x pior
Rate-burst — falhas	0,7%	0,9%	Estável
Idempotent replay — mediana	390ms	1.500ms	3,8x pior
Health check — mediana	130ms	180ms	Aceitável
Health check — falhas	3,3%	3,0%	Estável

Os mecanismos de resiliência **continuam funcionais** mesmo sob carga extrema: o rate limiting rejeita rajadas corretamente (taxa de falha funcional estável em ~0,7–0,9%), e o health check permanece relativamente responsivo (mediana 130–180ms). Porém, a vantagem do cache de idempotência é anulada pela fila de rede — com 100 usuários o replay já é mais rápido (390ms vs 470ms), mas com 1.000 usuários ambos convergem para ~1.500ms.

CEN-4 — Autenticação:

Teste	100 Usuários	1.000 Usuários	Veredicto
Sem auth (401)	0% falhas, 580ms	0% falhas, 1.600ms	APROVADO □
Auth inválida (403)	0% falhas, 590ms	0% falhas, 2.000ms	APROVADO □
Auth parcial (401)	0% falhas, 620ms	0% falhas, 1.700ms	APROVADO □

O protocolo de autenticação mantém **zero falhas funcionais** em ambos os cenários — todas as rejeições (401/403) ocorrem corretamente, independentemente da carga. A latência aumenta significativamente (3–4x), mas o comportamento de segurança é preservado.

CEN-5 — Não-Repúdio (`/api/v1/auditar/nr` + `/audit-log`):

Métrica	100 Usuários	1.000 Usuários	Veredicto
NR — mediana	1.100ms	2.400ms	REPROVADO
NR — falhas	18,9%	12,1%	REPROVADO
Audit-log — falhas	20,3%	16,0%	REPROVADO
NR — max	10.419ms	15.000ms	REPROVADO

O endpoint de não-repúdio é computacionalmente intensivo (HMAC-SHA256 + cadeia de hash) e apresenta a pior degradação relativa. O cálculo criptográfico sequencial se torna um gargalo sob concorrência, e o audit-log que cresce a cada requisição torna a verificação da cadeia progressivamente mais lenta (O(n)).

Diagnóstico: Causas-Raiz da Degradação

A análise comparativa revela 4 fatores principais:

1. **Worker único do Uvicorn:** a API roda com 1 processo e 1 thread. O event loop satura com ~50+ conexões simultâneas. Solução: `uvicorn --workers 4` ou Gunicorn com workers Uvicorn.

2. **Crescimento sublinear do RPS:** com 10x mais usuários (100 → 1.000), o throughput cresce apenas 3,9x (90 → 350 req/s), indicando saturação do servidor. O excedente vira fila TCP, inflando latências.

3. **Audit log em memória:** o log de auditoria (`CEN-5`) cresce com cada requisição. A verificação de integridade da cadeia de hash tem complexidade O(n), degradando progressivamente.

4. **Contenção de I/O síncrono:** operações criptográficas (HMAC, hash chain) executam de forma síncrona no event loop, bloqueando o processamento de outras requisições.

Conclusão e Vereditos

Cenário	100 Usuários	1.000 Usuários	Recomendação
DN1 SLA (P90≤500ms)	**REPROVADO**	**REPROVADO**	Escalar para múltiplos workers; otimizar processamento
DN3 Resiliência	**APROVADO**	**DEGRADADO**	Rate limiting funcional; latência precisa de escala
CEN-4 Auth	**APROVADO**	**APROVADO**	Segurança robusta em qualquer carga
CEN-5 Não-Repúdio	**REPROVADO**	**REPROVADO**	Mover criptografia para thread pool; limitar tamanho do log

O **ponto de ruptura funcional da API está entre 10 e 50 usuários concorrentes** em configuração single-worker. A partir de 100 usuários, o SLA de latência DN1 já é violado. Com 1.000 usuários, a degradação é generalizada, embora os mecanismos de segurança (autenticação) e resiliência funcional (rate limiting) permaneçam intactos — demonstrando que a arquitetura de proteção é sólida mesmo quando a capacidade computacional é insuficiente.

4.3 INT3 (Blue Team) — Integração de Gestão de Identidade com Clerk

Objetivo

Configurar o Clerk como Identity Provider (IDP) na aplicação, implementando autenticação híbrida em paralelo ao sistema existente de `account-key/app-key`. Este direcionador garante **isolamento de contas (DN4)** e assegura que o tratamento de dados de usuários esteja em total conformidade com a **LGPD** e padrões de segurança bancária.

Regras de Negócio

INT3.1 - Validação de Token JWT Clerk

- A API deve validar tokens JWT emitidos pelo Clerk usando JWKS (JSON Web Key Set) com algoritmo RS256
- Tokens expirados retornam HTTP 401 com mensagem `"INT3: Token JWT expirado"`
- Tokens com assinatura inválida retornam HTTP 401
- Requisições sem nenhum método de autenticação retornam HTTP 401 com referência explícita a `INT3`

INT3.2 - Isolamento de Contas por Tenant (DN4)

- Cada organização Clerk (`org_id`) é mapeada como um tenant isolado
- O `actor` de DN4 é derivado como `SHA-256(user_id:org_id)` para autenticação Clerk
- Um usuário de `org_A` não deve acessar dados, histórico ou log de auditoria de `org_B`
- O `tenant_id` é registrado em cada transação do `TransactionHistoryStore`

INT3.3 - Autenticação Híbrida (Retrocompatibilidade)

- O endpoint `POST /api/v1/auditar/clerk` aceita ambos os métodos de autenticação
- Header `Authorization: Bearer <token>` → Clerk JWT (OpenID Connect / OAuth 2.0)
- Headers `account-key` + `app-key` → API Keys legadas
- A resposta normaliza a identidade em um formato único com `auth_type`, `tenant_id` e `actor`

INT3.4 - Conformidade LGPD (Pseudonimização)

- Dados pessoais (email, nome) não são armazenados nos logs de auditoria
- O `user_id` original do Clerk não aparece em nenhuma resposta nem no `AuditLog`

- O `actor` é sempre um hash SHA-256 irreversível (pseudonimização, art. 12 LGPD)
- Apenas identificadores hasheados são persistidos no `TransactionHistoryStore`

INT3.5 - Cache de Chaves JWKS

- As chaves públicas do Clerk são cacheadas localmente com TTL de 1 hora
- O cache evita chamadas de rede desnecessárias, garantindo SLA de 500ms (DN1/RNF3)
- Falha na obtenção das chaves retorna HTTP 503

Especificação da API — Endpoint de Autenticação Híbrida

Endpoint principal: `POST /api/v1/auditar/clerk`

Aceita os mesmos campos de entrada dos demais endpoints de auditoria:

- `sessao_cliente`: Identificador único da sessão de auditoria.
- `valor`: Volume de documentos fiscais submetidos no lote.
- `destino`: Módulo fiscal de destino (ex: `EFD_ICMS_IPI`, `NCM_AUDIT`).

Headers aceitos (um dos dois métodos):

Método	Header(s)	Padrão
Clerk JWT	`Authorization: Bearer <token>`	OpenID Connect / OAuth 2.0
API Keys legado	`account-key: <key>` + `app-key: <key>`	Retrocompatibilidade

Resposta (HTTP 201):

```
{
  "id": 25000,
  "status": 201,
  "tracking_id": "uuid-v4",
  "arquivold": 22500,
  "diagnostico": "Conforme (Fallback Mock)",
  "detalhes": {
    "cliente": "sessao-int3-01",
    "volume_processado": 3200.0,
    "modulo": "EFD_ICMS_IPI"
  },
  "identity": {
    "auth_type": "clerk",
    "tenant_id": "org_alpha",
    "actor": "sha256-hex-irreversivel"
  },
  "non_repudiation": {
    "signature": "hmac-sha256-hex",
    "timestamp": "2026-03-12T22:00:00.000000+00:00",
    "actor": "sha256-hex-irreversivel",
    "receipt": {
      "signature": "...",
      "timestamp": "...",
      "actor": "...",
      "tracking_id": "...",
      "payload_hash": "..."
    },
    "log_position": 0,
    "log_entry_hash": "sha256-hex",
    "alert": null,
    "chain_alert": null
  }
}
```

Mapa de Direcionadores de Negócio — Blue Team (INT3)

Foram implementados 5 direcionadores que garantem a integração completa de gestão de identidade:

ID	Direcionador de Negócio	SLA / Critério de Aceite	Status
INT3.1	**Validação JWT Clerk (JWKS/RS256)**	100% dos tokens válidos aceitos; expirados, inválidos e ausentes rejeitados com HTTP 401.	**PASS**
INT3.2	**Isolamento de Tenant (DN4)**	`org_A` não acessa dados de `org_B`; actors distintos por organização; tenant_id no histórico.	**PASS**
INT3.3	**Autenticação Híbrida**	Clerk JWT e API Keys funcionam no mesmo endpoint; resposta normalizada com `auth_type`.	**PASS**
INT3.4	**Conformidade LGPD**	PII ausente em logs e respostas; actor = SHA-256(user_id:org_id); pseudonimização verificável.	**PASS**
INT3.5	**Integração com DN4 (Não-Repúdio)**	Recibo digital completo em ambos os métodos; transação registrada com `tenant_id`; cadeia de hash íntegra.	**PASS**

Implementação Técnica

Módulo de autenticação Clerk (`src/core/auth_clerk.py`):

```python
# INT3.1: Cache de JWKS com TTL (evita chamadas externas desnecessárias)
class JWKSCache:
    def get_keys(self) -> dict: ...      # Retorna chaves cacheadas ou bus
    def invalidate(self):  ...           # Invalida o cache (testes e rota

# INT3.1: Validação de token JWT usando JWKS do Clerk
def verify_clerk_token(token: str) -> dict: ...
    # Raises HTTPException 401 para token expirado ou inválido

# INT3.4: Extração de identidade com pseudonimização (LGPD)
def extract_clerk_identity(claims: dict) -> dict:
    # Retorna: auth_type, user_id, org_id, actor (SHA-256), tenant_id, ema

# INT3.2/DN4: Hash do ator para não-repúdio
def hash_clerk_actor(user_id: str, org_id: str) -> str:
    return hashlib.sha256(f"{user_id}:{org_id}".encode()).hexdigest()
```

Módulo de autenticação híbrida (`src/core/auth_hybrid.py`):

```python
# INT3.3: Orquestrador — detecta método de autenticação e normaliza identi
def authenticate_hybrid(
    authorization: Optional[str],  # Bearer token Clerk
    account_key: Optional[str],    # API Key legada
    app_key: Optional[str],        # API Key legada
) -> dict:
    # → Clerk JWT: chama verify_clerk_token() + extract_clerk_identity()
    # → API Keys: valida contra VALID_CREDENTIAL_PAIRS, gera actor=SHA-256
    # → Nenhum: HTTP 401 com orientação sobre ambos os métodos
```

Endpoint híbrido (`src/core/api.py`):

```python
# INT3 — Endpoint de auditoria com autenticação híbrida
@app.post("/api/v1/auditar/clerk", status_code=201)
async def processar_auditoria_clerk(payload, authorization, account_key, a
    # Reutiliza validações DN2, DN4 e RF2
    # actor derivado da identidade híbrida (Clerk ou API Key)
    # tenant_id registrado no TransactionHistoryStore
```

Metodologia de Testagem (Blue Team — TDD)

A aferição foi conduzida utilizando abordagem Test Data Driven (TDD) através do script `src/tests/blue_team/INT3_clerk_auth_blue.py`.

Cada cenário de teste é executado 3 vezes (iterações parametrizadas via `@pytest.mark.parametrize`) para garantir consistência e reprodutibilidade dos resultados.

Massa de testes: Os payloads simulam sessões reais de auditoria fiscal com volumes variados (`PAYLOAD_VALIDO`: 3200.00 docs para EFD_ICMS_IPI; `PAYLOAD_ALTERNATIVO`: 8100.75 docs para NCM_AUDIT), exercitando ambos os métodos de autenticação.

Versão da API testada: v2.0.0 — `POST /api/v1/auditar/clerk`

Isolamento de chamadas externas: O cache JWKS é substituído por `_MockJWKSCache` e o `jwt.decode` é mockado via `unittest.mock.patch`, garantindo que nenhuma chamada de rede real ao Clerk ocorra durante a suíte.

Estrutura dos testes (5 classes, 20 cenários × 3 iterações = 60 execuções):

Classe de Teste	Cenários	Direcionador
`TestINT3JWTValidation`	5 testes × 3 iter	INT3.1
`TestINT3TenantIsolation`	3 testes × 3 iter	INT3.2
`TestINT3HybridAuthentication`	4 testes × 3 iter	INT3.3
`TestINT3LGPDCompliance`	4 testes × 3 iter	INT3.4
`TestINT3NonRepudiationIntegration`	4 testes × 3 iter	INT3.5

Resultados Obtidos

A suíte de testes validou a implementação do INT3 com os seguintes indicadores:

- **Status Final:** APROVADO □
- **Taxa de Cobertura:** 100% (20 cenários × 3 iterações = 60 execuções passando)
- **Cenários Testados:**
  - 15 cenários de validação JWT (INT3.1)
  - 9 cenários de isolamento de tenant (INT3.2)
  - 12 cenários de autenticação híbrida (INT3.3)
  - 12 cenários de conformidade LGPD (INT3.4)
  - 12 cenários de integração com DN4 (INT3.5)
- **Verificações de Negócio:**
  - INT3.1: Token válido aceito, expirado/inválido/ausente rejeitados → 5 testes □
  - INT3.2: Actors distintos por tenant, histórico isolado → 3 testes □
  - INT3.3: Clerk e API Key no mesmo endpoint, resposta normalizada → 4 testes □
  - INT3.4: PII ausente em logs, actor=SHA-256 verificável → 4 testes □
  - INT3.5: Recibo digital e cadeia de hash íntegros em ambos os métodos → 4 testes □

Execução dos Testes

```
# Instalar dependências (inclui PyJWT e python-jose para validação JWT)
pip install -r requirements.txt

# Executar todos os testes INT3 (Blue Team)
pytest src/tests/blue_team/INT3_clerk_auth_blue.py -v

# Executar por classe de teste
pytest src/tests/blue_team/INT3_clerk_auth_blue.py::TestINT3JWTValidation
pytest src/tests/blue_team/INT3_clerk_auth_blue.py::TestINT3TenantIsolatio
pytest src/tests/blue_team/INT3_clerk_auth_blue.py::TestINT3HybridAuthenti
pytest src/tests/blue_team/INT3_clerk_auth_blue.py::TestINT3LGPDCompliance
pytest src/tests/blue_team/INT3_clerk_auth_blue.py::TestINT3NonRepudiation
```

Decisão Arquitetural e Contexto de BD/DN

A escolha do **modelo de autenticação híbrida** — mantendo API Keys para integração M2M e adicionando Clerk JWT para usuários e aplicações modernas — decorre de um risco de negócio identificado pelo Red Team (DN4.R2 — Header Impersonation, **FAIL**): o sistema atual não valida autenticidade real de credenciais, comprometendo o mecanismo de não-repúdio (DN4). A correção via Clerk resolve esse gap sem quebrar retrocompatibilidade com integrações existentes.

A decisão de derivar o `actor` como `SHA-256(user_id:org_id)` para Clerk — em vez de apenas `SHA-256(user_id)` — é intencional: ela vincula cada transação não apenas ao usuário, mas à **organização (tenant)** à qual pertence, garantindo que o isolamento de contas (DN4) seja rastreável de forma criptograficamente verificável. Isso é especialmente relevante no contexto bancário e fiscal, onde auditorias externas podem exigir prova de que determinada transação pertence a uma conta específica.

O **cache JWKS com TTL de 1 hora** é uma decisão de desempenho derivada diretamente do SLA de 500ms (DN1/RNF3): sem cache, cada requisição JWT exigiria uma chamada HTTP ao endpoint do Clerk, adicionando latência de rede incompatível com o SLA estabelecido.

Configuração de Variáveis de Ambiente

A integração Clerk requer três variáveis de ambiente configuradas no arquivo `.env` (use `.env.example` como referência):

```
# Como obter as chaves:
# 1. Crie uma conta em https://clerk.com e um novo Application.
# 2. Em "API Keys" do dashboard, copie as chaves abaixo.
# 3. CLERK_JWKS_URL: disponível em Settings > API Keys > "Show JWKS URL"

CLERK_PUBLISHABLE_KEY=pk_test_seu_valor_aqui
CLERK_SECRET_KEY=sk_test_seu_valor_aqui
CLERK_JWKS_URL=https://seu-dominio.clerk.accounts.dev/.well-known/jwks.jso
```

Impacto na Qualidade — Aspectos Positivos e Negativos Mapeados

Aspectos positivos:

- **Resolução do gap de autenticidade (RNF2/DN4):** A integração Clerk elimina a vulnerabilidade de Header Impersonation identificada pelo Red Team (DN4.R2 — FAIL), tornando o `actor` de não-repúdio criptograficamente vinculado a uma identidade verificada por um IDP externo.
- **Isolamento de contas robusto (DN4):** O mapeamento Clerk Organization → tenant garante que cada cliente opera em um espaço isolado, com histórico de transações, rate limiting e audit log segregados por `tenant_id`.
- **Conformidade LGPD por design:** A pseudonimização SHA-256 de todos os identificadores pessoais é aplicada na camada de extração de claims, garantindo que nenhuma PII chegue a logs ou respostas — arquitetura *privacy by design*.
- **Retrocompatibilidade total:** O modelo híbrido garante que integrações M2M

existentes (API Keys) continuem funcionando sem modificações, reduzindo o risco de migração.
- **Base para padrões bancários:** A adoção de OpenID Connect / OAuth 2.0 com IDP externo é o padrão do mercado financeiro (Open Banking, FAPI), criando fundação para futuras certificações de segurança.

Aspectos negativos / fora do foco atual:

- A validação JWT depende de conectividade com o endpoint JWKS do Clerk durante a inicialização (ou na primeira requisição após expiração do cache). Em ambientes totalmente isolados (air-gapped), seria necessário pré-carregar as chaves.
- O modelo atual não implementa revogação de token em tempo real (token blacklist) — um token válido mas comprometido permaneceria aceito até sua expiração natural. Em produção, recomenda-se integração com o endpoint de revogação do Clerk.
- O `CLERK_SECRET_KEY` presente no `.env` não é utilizado na validação JWT (que usa JWKS público), mas é necessário para operações administrativas via Clerk Backend API. Deve ser tratado como segredo e nunca versionado.
- Não foram implementados testes Red Team para INT3 (ex.: ataques de token replay, JWK confusion, algorithm confusion). Ficam como recomendação para a próxima sprint.

Justificativa de priorização: A integração INT3 foi priorizada como primeira integração externa da sprint por ser pré-requisito para as demais: sem autenticidade real, os mecanismos de isolamento (DN4), não-repúdio e conformidade LGPD perdem fundamento. A escolha do Clerk em substituição ao Auth0 original decorre de menor complexidade de configuração inicial e SDK mais direto para validação JWT server-side em Python.

4.4 INT4 — Sincronismo de versões via API do GitLab

Sprint 3 – Tarefa 4. Serviço de orquestração que utiliza a API do GitLab para verificar se a versão em produção corresponde à versão homologada (governança do SDLC).

4.4.1 Assinatura / critério de versão

Item	Valor
API	GitLab REST API **v4**
Recurso	Commit/Tag Sync (commits por branch)
Endpoint utilizado	`GET /projects/:id/repository/commits?ref_name=<branch>&per_page=1`
Documentação oficial	GitLab Commits API

4.4.2 Objetivo da integração

- Comparar automaticamente o **último commit da branch** `main` (produção) com o **último commit da branch** `develop` (homologação).
- Retornar status **em sync** ou **out of sync**, permitindo auditoria e governança do ciclo de vida do software (SDLC).
- Função **`verificar_protocolo_gitlab()`** confirma a conformidade do protocolo da integração (API v4 acessível e formato de resposta esperado), ou indica mudanças.

4.4.3 Requisitos e DNs relacionados

- **Sprint 3 (INT4):** Verificar se a versão em Produção é a mesma homologada.
- **Governança SDLC:** Rastreabilidade entre código homologado e código em produção.
- **Documento de referência:** `docs/gestao-configuração.md` (política de branches: `main` = produção, `develop` = integração contínua).

4.4.4 Como se conectar ao serviço

1. Variáveis de ambiente

Configure na raiz do projeto (arquivo `.env`, **nunca** commitado):

Variável	Obrigatório	Descrição
`GITLAB_PRIVATE_TOKEN`	Sim	Token de acesso (Personal Access Token ou Project Access Token) com permissão `read_api` e `read_repository`.
`GITLAB_BASE_URL`	Não	Base URL do GitLab. Padrão: `https://git.inteli.edu.br`.
`GITLAB_PROJECT_PATH`	Não	Path do projeto (ex.: `graduacao/2026-1a/t13/g06`). Padrão: repositório do projeto.

2. Criação do token no GitLab

1. Acesse o GitLab (ex.: https://git.inteli.edu.br).
2. **Settings → Access Tokens** (ou no projeto: **Settings → Access Tokens**).
3. Crie um token com escopo **read_api** e **read_repository**.
4. Copie o valor e defina em `GITLAB_PRIVATE_TOKEN` no `.env`.

3. Execução do script

Na raiz do repositório:

```
# Saída legível (status + commits)
python scripts/verifica_versao_gitlab.py

# Saída em JSON (para cron/automação)
python scripts/verifica_versao_gitlab.py --json

# Apenas verificação do protocolo da API (não compara commits)
python scripts/verifica_versao_gitlab.py --apenas-protocolo
```

Exit code: `0` quando produção e homologação estão na mesma versão; `1` quando estão diferentes ou há erro.

4. Execução periódica (cron)

Exemplo para rodar a cada hora e registrar o resultado:

```
0 * * * * cd /caminho/para/repo && python scripts/verifica_versao_gitlab.p
```

4.4.5 Função de verificação do protocolo

A função **`verificar_protocolo_gitlab()`** (em `scripts/verifica_versao_gitlab.py`):

1. Chama `GET /projects/:id` para validar acesso ao projeto e que a API está respondendo.
2. Chama `GET /projects/:id/repository/commits?ref_name=main&per_page=1` e valida que a resposta é uma lista de commits com campos `id` e `title`.

Retorno: `{"ok": true/false, "message": "...", "details": {...}}`. Se `ok` for `false`, a mensagem indica token inválido, projeto não encontrado ou mudança no formato da API (protocolo).

4.4.6 Testes

Testes de integração em `src/tests/blue_team/gitlab_sync_blue.py` que chamam a **API real do GitLab** (sem mock). É necessário ter `GITLAB_PRIVATE_TOKEN` válido no `.env` na raiz do projeto.

```
pytest src/tests/blue_team/gitlab_sync_blue.py -v
```

Os testes validam: estrutura do retorno de `verificar_protocolo_gitlab()`, obtenção do último commit das branches `main` e `develop`, comparação de sincronismo e serialização JSON. Se o token não estiver configurado (ou for um valor de teste), os testes são ignorados (`skip`).

4.5 INT5 (Blue Team) — Aferição de Layout SPED/RFB

Objetivo

A INT5 implementa um motor de **aferição de conformidade de layout** que valida se os payloads gerados pelo sistema ASIS e destinados ao SPED estão em rigorosa conformidade com o **Layout Oficial (v2026)** da Receita Federal. A validação é autocontida — não depende de chamadas de rede externas ou da disponibilidade da infraestrutura da RFB.

Assinatura / Critério de Versão

Item	Valor
Serviço	RFB / SPED (Layout Fiscal)
Versão do Layout	`v2026`
Tipo de integração	Externa (simulada localmente)
Módulo	`src/tests/blue_team/test_int5_sped_afericao.py`

Regras de Validação do Layout

O motor valida o payload contra a definição `LAYOUT_RF1_V2026`:

Campo	Tipo	Obrigatório	Regra Adicional
`versao`	`str`	Sim	Valor exato: `"v2026"`
`cnpj_emissor`	`str`	Sim	Exatamente 14 caracteres
`competencia`	`str`	Sim	—
`hash_lote`	`str`	Sim	—
`volume_transacoes`	`int`	Não	—

Metodologia de Testagem (Blue Team)

A suíte em `src/tests/blue_team/test_int5_sped_afericao.py` utiliza `TestClient` para enviar requisições à API ASIS, intercepta o campo `payload_sped_gerado` da resposta e valida esse contrato contra o layout oficial.

Estrutura dos testes (3 cenários):

Cenário	Classe	Descrição
INT5.B1	`TestINT5SpedAfericaoLayoutIntegrado`	Payload ASIS válido gera contrato CONFORME ao layout v2026
INT5.B2	`TestINT5SpedAfericaoLayoutIntegrado`	Simulação de campo novo exigido pela Receita detecta QUEBRA DE CONTRATO
INT5.B3	`TestINT5SpedAfericaoLayoutIntegrado`	Estrutura do relatório de aferição contém campos obrigatórios (`integracao`, `timestamp`, `versao_layout`, `status`)

Resultados Obtidos

Todos os 3 cenários **PASS**. O motor de aferição:

- Confirma conformidade quando o payload ASIS segue o layout v2026
- Detecta e reporta quebras de contrato quando o layout evolui (campo novo obrigatório)
- Gera relatórios estruturados com `integracao`, `timestamp`, `versao_layout` e `status`

Execução dos Testes

```
# Executar testes INT5 Blue Team
pytest src/tests/blue_team/test_int5_sped_afericao.py -v -s
```

Relação com Business Drivers

Driver	Como a INT5 contribui
RF1 (Processamento Assíncrono)	Garante que o payload gerado pelo endpoint de auditoria segue o layout fiscal exigido
DN2 (Integridade)	Valida a correção estrutural do contrato SPED antes de eventual envio à RFB
TAP (Replicabilidade)	Motor de validação desacoplado que pode ser reutilizado para outros layouts fiscais

5. Direcionadores Adversariais — Red Team (Sprint de Consolidação)

Esta seção documenta os testes adversariais (Red Team) implementados para os componentes que anteriormente possuíam apenas cobertura Blue Team. O Red Team simula cenários de ataque para expor vulnerabilidades, limites e lacunas que os testes de aceite convencionais não exercitam.

5.1 INT3 (Red Team) — Autenticação Híbrida Clerk sob Perspectiva Adversarial

Objetivo do Ativo Adversarial

Validar se a integração de identidade com Clerk (JWT + API Keys) resiste a cenários de ataque que visam **bypass de autenticação**, **confusão de algoritmo**, **adulteração de claims**, **vazamento de PII** e **confusão de método híbrido**. O Red Team tenta explorar as fronteiras entre os dois métodos de autenticação e verificar se a conformidade LGPD se mantém sob condições adversas.

Mapa de Direcionadores de Negócio - Red Team (INT3)

ID	Cenário de Ataque	Critério de Ataque	Status
INT3.R1	**Token Replay**	Reuso de JWT idêntico — verificar se existe controle anti-replay (nonce/jti)	**PASS**
INT3.R2	**Algorithm Confusion**	Tokens com `alg:none` ou HS256 quando RS256 é esperado — devem ser rejeitados (401)	**PASS**
INT3.R3	**Claims Tampering**	JWT com `org_id` adulterado gera actor diferente, impedindo colisão de identidade cross-tenant	**PASS**
INT3.R4	**Tenant Boundary Crossing**	Tenant A (Clerk) cria transação; Tenant B (API Key) tenta acessar histórico — bloqueado com 403	**PASS**
INT3.R5	**LGPD Information Leakage**	Verificar ausência de PII em respostas de erro, audit log e health check	**PASS**
INT3.R6	**Hybrid Auth Confusion**	Envio simultâneo de Bearer + API Keys — sistema escolhe método sem erro 500	**PASS**
INT3.R7	**Malformed JWT**	Tokens sem segmentos, base64 inválido, payload gigante (100KB) — rejeitados com 401	**PASS**

Métodos de Testagem

A suíte em `src/tests/red_team/INT3_clerk_auth_red.py` utiliza `TestClient` in-process com mocks de `jwt.decode` e `JWKSCache` para simular cenários adversariais sem dependência de rede. Cada classe de teste corresponde a um cenário de ataque (INT3.R1 a INT3.R7).

Estrutura dos testes (7 classes, 16 cenários):

Classe de Teste	Cenários	Direcionador
`TestINT3R1TokenReplay`	2 testes	INT3.R1
`TestINT3R2AlgorithmConfusion`	2 testes	INT3.R2
`TestINT3R3ClaimsTampering`	2 testes	INT3.R3
`TestINT3R4TenantBoundary`	1 teste	INT3.R4
`TestINT3R5LGPDLeakage`	3 testes	INT3.R5
`TestINT3R6HybridConfusion`	2 testes	INT3.R6
`TestINT3R7MalformedJWT`	4 testes	INT3.R7

Execução dos Testes

```
# Via pytest (in-process)
pytest src/tests/red_team/INT3_clerk_auth_red.py -v -s

# Contra API real
RED_TEAM_API_URL=http://127.0.0.1:8000 pytest src/tests/red_team/INT3_cler
```

Análise dos Resultados

Pontos Fortes:

- A validação JWT rejeita corretamente tokens com algoritmo inválido (`alg:none`, HS256) e tokens malformados (segmentos ausentes, base64 corrompido, payload gigante).
- O isolamento de tenant funciona: Clerk Tenant A não acessa dados de Tenant B via `/historico/`.
- Nenhuma PII (email, user_id original) vaza em respostas de erro ou no audit log (conformidade LGPD mantida).
- A autenticação híbrida trata envio simultâneo de Bearer + API Keys sem erro 500.

Pontos de Quebra Identificados:

- O sistema não possui controle anti-replay por jti/nonce — o mesmo token pode ser reusado até expirar. Mitigação parcial pelo cache de replay por payload.
- Tokens com `sub` vazio são aceitos (201) — em produção, validação adicional de claims obrigatórios seria recomendada.

Síntese de Impacto (Positivos x Negativos) e Priorizações

Aspectos positivos:

- A arquitetura de validação JWT está resiliente aos ataques mais comuns (algorithm confusion, malformed tokens, claims tampering).
- A pseudonimização LGPD se mantém robusta sob condições adversas — nenhum cenário conseguiu extrair PII das respostas.
- O isolamento de tenant é efetivo tanto para autenticação Clerk quanto para API Keys.

Aspectos negativos:

- Ausência de controle anti-replay por jti/nonce no JWT — recomendação: implementar blacklist de token por `jti` com TTL igual ao `exp`.
- Claims com campos vazios (`sub: ""`) não são rejeitados — recomendação: validar presença de `sub` e `org_id` não vazios.
- Não foram testados cenários de JWK confusion com chaves de outro IDP (rotação de chaves comprometida).

Priorização: Implementar blacklist de jti para anti-replay (alta) > Validar claims obrigatórios (média) > Testes de JWK rotation (baixa).

5.2 RNF1 (Red Team) — Disputa de Não-Repúdio sob Perspectiva Adversarial

Objetivo do Ativo Adversarial

Validar se o motor de sazonalidade e o endpoint de disputa resistem a cenários de **envenenamento de histórico**, **disputas cross-tenant**, **flooding**, **enumeração de tracking_ids**, **manipulação temporal** e **adulteração de recibos**. O Red Team tenta manipular os dados de entrada para forçar vereditos incorretos e explorar limites do sistema de análise de disputas.

Mapa de Direcionadores de Negócio - Red Team (RNF1)

ID	Cenário de Ataque	Critério de Ataque	Status
RNF1.R1	**History Poisoning**	Criar histórico enviesado (valores baixos) e disputar transação de valor extremo — detecção de anomalia	**PASS**
RNF1.R2	**Cross-Tenant Dispute**	Tenant B tenta disputar transação de Tenant A — deve ser bloqueado ou isolado	**PASS**
RNF1.R3	**Dispute Flooding**	20 disputas consecutivas sobre a mesma transação — sem erro 500, respostas consistentes	**PASS**
RNF1.R4	**Tracking_id Enumeration**	10 tracking_ids fictícios — todos retornam 404 genérico sem informação sensível	**PASS**
RNF1.R5	**Temporal Manipulation**	Disputa com `data_reclamacao` no futuro (2099) e no passado (2005) — tratamento sem crash	**PASS**
RNF1.R6	**Verdict Bypass**	Payloads crafted para forçar vereditos específicos — veredito coerente com análises	**PASS**
RNF1.R7	**Receipt Tampering Detection**	Adulteração de hash na cadeia de auditoria — detecção de integridade comprometida	**PASS**

Métodos de Testagem

A suíte em `src/tests/red_team/RNF1_non_repudiation_dispute_red.py` opera via `TestClient` in-process, criando transações via `/auditar/nr` e abrindo disputas via `/disputa/nr`.

Estrutura dos testes (7 classes, 12 cenários):

Classe de Teste	Cenários	Direcionador
`TestRNF1R1HistoryPoisoning`	2 testes	RNF1.R1
`TestRNF1R2CrossTenantDispute`	2 testes	RNF1.R2
`TestRNF1R3DisputeFlooding`	1 teste	RNF1.R3
`TestRNF1R4TrackingIdEnumeration`	1 teste	RNF1.R4
`TestRNF1R5TemporalManipulation`	2 testes	RNF1.R5
`TestRNF1R6VerdictBypass`	2 testes	RNF1.R6
`TestRNF1R7ReceiptTampering`	2 testes	RNF1.R7

Execução dos Testes

```
pytest src/tests/red_team/RNF1_non_repudiation_dispute_red.py -v -s
```

Análise dos Resultados

Pontos Fortes:

- O motor de sazonalidade detecta corretamente valores anômalos mesmo quando o histórico é enviesado com transações de valor baixo.
- O endpoint de disputa mantém respostas consistentes sob flooding (20 disputas consecutivas sem degradação).
- Tracking_ids fictícios retornam 404 genérico sem vazamento de informação sobre processos existentes.
- A cadeia de hash detecta adulteração de recibos corretamente (`chain_integrity: False`).

Pontos de Quebra Identificados:

- O endpoint de disputa não possui rate limiting próprio — flooding de disputas é possível sem restrição.
- Disputas cross-tenant não são bloqueadas explicitamente por isolamento — o sistema

retorna análise sobre a transação independente do tenant solicitante.
- Datas de reclamação no futuro ou passado distante são aceitas sem validação temporal.

Síntese de Impacto e Priorizações

Priorização: Rate limiting no endpoint de disputa (alta) > Isolamento de tenant em disputas (alta) > Validação de data_reclamacao (média).

5.3 RNF2 (Red Team) — Autenticidade sob Perspectiva Adversarial

Objetivo do Ativo Adversarial

Validar se o mecanismo de autenticação por credenciais (`account-key`/`app-key`) resiste a **credential stuffing**, **timing side-channel**, **injeção de headers**, **bypass por encoding** e **vazamento de informação em mensagens de erro**.

Mapa de Direcionadores de Negócio - Red Team (RNF2)

ID	Cenário de Ataque	Critério de Ataque	Status
RNF2.R1	**Credential Stuffing**	50 combinações aleatórias de credenciais — todas rejeitadas (403), nenhuma aceita	**PASS**
RNF2.R2	**Timing Side-Channel**	Diferença de tempo entre 401/403/201 < 1 segundo — sem informação útil para atacante	**PASS**
RNF2.R3	**Header Injection**	CRLF, null bytes e newlines em `account-key` — rejeitados (401/403)	**PASS**
RNF2.R4	**Case/Encoding Bypass**	Credenciais com case diferente, espaços e homoglifos Unicode — rejeitadas corretamente	**PASS**
RNF2.R5	**Missing Header Variations**	Combinações parciais de headers (só account-key, só app-key, vazios, nomes errados) — 401	**PASS**
RNF2.R6	**Cross-Endpoint Credential Reuse**	Credenciais válidas funcionam em todos endpoints protegidos; health é público	**PASS**
RNF2.R7	**Error Message Information Leakage**	Mensagens de 401/403 idênticas independente de qual credencial é inválida — sem information leakage	**PASS**

Métodos de Testagem

A suíte em `src/tests/red_team/RNF2_authenticity_red.py` opera via `TestClient` e suporta execução contra API real via `RED_TEAM_API_URL`.

Estrutura dos testes (7 classes, 20 cenários):

Classe de Teste	Cenários	Direcionador
`TestRNF2R1CredentialStuffing`	2 testes	RNF2.R1
`TestRNF2R2TimingSideChannel`	1 teste	RNF2.R2
`TestRNF2R3HeaderInjection`	4 testes	RNF2.R3
`TestRNF2R4CaseEncodingBypass`	3 testes	RNF2.R4
`TestRNF2R5MissingHeaders`	4 testes	RNF2.R5
`TestRNF2R6CrossEndpoint`	3 testes	RNF2.R6
`TestRNF2R7ErrorLeakage`	3 testes	RNF2.R7

Execução dos Testes

```
pytest src/tests/red_team/RNF2_authenticity_red.py -v -s
```

Análise dos Resultados

Pontos Fortes:

- O mecanismo de autenticação rejeita corretamente todas as 50 tentativas de credential stuffing sem degradação.
- As mensagens de erro são idênticas para `account-key` inválida vs `app-key` inválida, impedindo enumeração de credenciais válidas.
- Headers com CRLF injection e null bytes são tratados sem efeito colateral.
- O timing entre cenários de autenticação é uniforme (< 1s de diferença), dificultando ataques de side-channel.

Pontos de Quebra Identificados:

- Credenciais com espaços em volta podem ser aceitas (trim) — comportamento a validar em produção.
- Headers HTTP são restritos a ASCII (RFC 7230), o que protege contra homoglifos Unicode na camada de transporte.
- Não há mecanismo de lockout após múltiplas tentativas falhas — credential stuffing prolongado não é limitado.

Síntese de Impacto e Priorizações

Priorização: Lockout após N tentativas falhas (alta) > Trim/sanitização de credenciais (média) > Rate limiting por IP (média).

5.4 INT4 (Red Team) — Sincronismo GitLab sob Perspectiva Adversarial

Objetivo do Ativo Adversarial

Validar se a integração com a API do GitLab (INT4) resiste a **path traversal**, **injeção em nomes de branch**, **respostas malformadas**, **rate limiting** e **timeout**. O Red Team simula cenários onde a API do GitLab retorna dados inesperados ou onde os parâmetros de entrada são manipulados.

Mapa de Direcionadores de Negócio - Red Team (INT4)

ID	Cenário de Ataque	Critério de Ataque	Status
INT4.R1	**Path Traversal**	Injeção de `../` e caracteres especiais no `GITLAB_PROJECT_PATH` — sem acesso a outros projetos	**PASS**
INT4.R2	**Branch Injection**	Nomes de branch com `;`, `\x00`, SQL injection, espaços — tratados sem crash	**PASS**
INT4.R3	**Response Validation Robustness**	JSON inválido, lista vazia, campos ausentes, dict em vez de lista — tratados sem crash	**PASS**
INT4.R4	**Token Scope Verification**	Funções operam apenas com escopos de leitura (read_api) — sem operações de escrita	**PASS**
INT4.R5	**API Rate Limit / Timeout Handling**	HTTP 429 e timeout de conexão — tratados graciosamente, retornando None/ok=False	**PASS**

Métodos de Testagem

A suíte em `src/tests/red_team/gitlab_sync_red.py` utiliza `monkeypatch` e `unittest.mock` para simular respostas adversariais da API do GitLab. Testes de escopo (R4) requerem `GITLAB_PRIVATE_TOKEN` real.

Estrutura dos testes (5 classes, 18 cenários):

Classe de Teste	Cenários	Direcionador
`TestINT4R1PathTraversal`	2 testes	INT4.R1
`TestINT4R2BranchInjection`	6 testes (parametrizados)	INT4.R2
`TestINT4R3ResponseValidation`	5 testes	INT4.R3
`TestINT4R4TokenScope`	2 testes (requer token)	INT4.R4
`TestINT4R5RateLimitHandling`	3 testes	INT4.R5

Execução dos Testes

```
# Sem token (testes com mock apenas)
pytest src/tests/red_team/gitlab_sync_red.py -v -s

# Com token (testes de escopo incluídos)
GITLAB_PRIVATE_TOKEN=glpat-xxx pytest src/tests/red_team/gitlab_sync_red.p
```

Análise dos Resultados

Pontos Fortes:

- Path traversal e branch injection não causam crash nem acesso indevido — as funções retornam `None` para parâmetros maliciosos.
- Respostas malformadas da API GitLab (JSON inválido, formato inesperado) são tratadas graciosamente.
- Rate limiting (429) e timeout são capturados e retornam valores de falha controlados.
- As funções de integração realizam apenas operações de leitura (GET), sem risco de modificação de dados.

Pontos de Quebra Identificados:

- O script original (`verifica_versao_gitlab.py`) possui um `except` com tupla aninhada (`except (RequestException, (KeyError, TypeError))`) que em Python 3.14 gera `TypeError`. Recomenda-se corrigir para `except (RequestException, KeyError, TypeError)`.
- Não há sanitização explícita dos parâmetros `GITLAB_PROJECT_PATH` e `ref_name` antes de montar a URL — a API do GitLab faz o URL-encoding, mas validação adicional seria recomendada.

Síntese de Impacto e Priorizações

Priorização: Corrigir except aninhado no script (alta) > Sanitização de parâmetros de entrada (média) > Retry com backoff para 429 (baixa).

5.5 INT5 (Red Team) — Aferição SPED/RFB sob Perspectiva Adversarial

Objetivo do Ativo Adversarial

Validar se o motor de validação de layout SPED e o endpoint de auditoria resistem a **spoofing de versão**, **injeção em campos CNPJ**, **manipulação de hash**, **payloads oversized**, **edge cases de competência** e **combinações de campos ausentes**. O Red

Team tenta explorar limites do validador de layout para aceitar payloads inválidos ou rejeitar payloads válidos.

Mapa de Direcionadores de Negócio - Red Team (INT5)

ID	Cenário de Ataque	Critério de Ataque	Status
INT5.R1	**Layout Version Spoofing**	7 versões inválidas (v2025, v2027, V2026, "", etc.) — todas rejeitadas pelo validador	**PASS**
INT5.R2	**CNPJ Injection**	SQL injection, XSS, null byte, tipo incorreto — rejeitados por tamanho ou tipo	**PASS**
INT5.R3	**Hash Manipulation**	Hash vazio, tipo incorreto, ausente — tipo e presença validados	**PASS**
INT5.R4	**Oversized Payload**	5.000 campos extras, string de 1MB, payload pesado na API — sem crash, < 5s	**PASS**
INT5.R5	**Competência Edge Cases**	Datas futuras, passadas, formato inválido, tipo incorreto — aceitas (gap documentado)	**PASS**
INT5.R6	**Missing Fields Combinatorial**	Todas as combinações de campos obrigatórios ausentes — corretamente detectadas	**PASS**

Métodos de Testagem

A suíte em `src/tests/red_team/INT5_sped_afericao_red.py` testa diretamente as funções `validar_layout_sped` e `aferir_layout_int5` do módulo de aferição INT5, além de testar a API via `TestClient`.

Estrutura dos testes (6 classes, 38 cenários):

Classe de Teste	Cenários	Direcionador
`TestINT5R1VersionSpoofing`	8 testes (parametrizados)	INT5.R1
`TestINT5R2CNPJInjection`	7 testes	INT5.R2
`TestINT5R3HashManipulation`	3 testes	INT5.R3
`TestINT5R4OversizedPayload`	3 testes	INT5.R4
`TestINT5R5CompetenciaEdgeCases`	9 testes (parametrizados)	INT5.R5
`TestINT5R6MissingFieldsCombinatorial`	8 testes	INT5.R6

Execução dos Testes

```
pytest src/tests/red_team/INT5_sped_afericao_red.py -v -s
```

Análise dos Resultados

Pontos Fortes:

- A validação de versão é estrita: apenas `"v2026"` é aceita; variantes com case diferente (`V2026`), sem prefixo (`2026`), null byte (`v2026\x00`) e versões adjacentes são corretamente rejeitadas.
- CNPJ com SQL injection, XSS e tamanhos incorretos são rejeitados pela validação de tamanho (14 caracteres) e tipo.
- Payloads com 5.000 campos extras são processados em < 5 segundos sem crash — campos desconhecidos são ignorados pelo validador.
- Todas as combinações de campos obrigatórios ausentes (individuais e em pares) são corretamente detectadas.

Pontos de Quebra Identificados:

- **CNPJ com null byte:** String `"1234567800019\x00"` tem `len() == 14` e é aceita pelo validador, pois a validação é apenas por tamanho. Recomendação: sanitizar null bytes antes da validação de tamanho.

- **Competência sem validação semântica:** O campo `competencia` aceita qualquer string (ex.: `"2099-12"`, `"invalido"`, `""`). Recomendação: implementar validação de formato (YYYY-MM) e faixa temporal.
- **Hash vazio aceito:** O campo `hash_lote` com string vazia (`""`) é aceito pois está presente e é do tipo correto. Recomendação: adicionar validação de conteúdo mínimo.

Síntese de Impacto e Priorizações

Priorização: Sanitização de null bytes em CNPJ (alta) > Validação semântica de competência (alta) > Validação de hash não vazio (média) > Validação de dígitos verificadores do CNPJ (baixa).

---

6. Dashboard e Observabilidade

6.1 Prometheus

A API ASIS TaxTech integra o **Prometheus** para coleta de métricas alinhadas aos Direcionadores de Negócio (DN1–DN4), Requisitos Funcionais (RF1, RF2) e Requisitos Não Funcionais (RNF1). O Prometheus faz scrape periódico dos endpoints de métricas e alimenta regras de alerta e dashboards (ex.: Grafana).

Endpoints definidos

Endpoint	Porta	Descrição
`GET /metrics`	8000	Métricas Prometheus da API ASIS (latência, requisições, health, circuit breaker, rate limit, audit log). Formato texto Prometheus, sem autenticação.
`GET /metrics`	9092	Exporter de métricas pytest (`pytest_tests_total`, `pytest_tests_passed`). Servidor em `src/scripts/pytest_metrics_exporter.py`.

Métricas expostas (DN1–DN4, RF1, RNF1)

Categoria	Métricas	Uso
DN1 Latência	`asis_request_duration_seconds`, `asis_requests_total`	Histograma e contador por endpoint, method, status, source
DN2 Integridade	`asis_validation_errors_total`	Erros de validação (HTTP 400)
DN3 Disponibilidade	`asis_health_uptime_seconds`, `asis_health_error_rate_percent`, `asis_circuit_breaker_state`, `asis_rate_limit_remaining`	Health, circuit breaker, rate limit
DN4 Não-Repúdio	`asis_audit_log_entries_total`, `asis_audit_log_size`, `asis_nr_alerts_total`	Log de auditoria e alertas
Registros	`asis_process_store_size`, `asis_idempotency_store_size`	Tamanho dos stores
Capacidade	`asis_concurrent_requests`	Requisições concorrentes

Referência completa: `docs/catalogo_servicos_monitorados.md`.

Passos para rodar os servidores

Ordem recomendada:

1. **API ASIS** (porta 8000):

   ```
   cd g06
   uvicorn src.core.api:app --host 0.0.0.0 --port 8000
   ```

2. **Exporter pytest** (porta 9092, opcional):

   ```
   python src/scripts/pytest_metrics_exporter.py
   ```

3. **Prometheus e Alertmanager** (portas 9090 e 9093): via Docker Compose (ver seção seguinte).

Docker Compose

O stack Prometheus + Alertmanager sobe com:

```
cd g06
docker-compose up -d
```

- **Prometheus:** http://localhost:9090 (targets, graph, alerts)
- **Alertmanager:** http://localhost:9093

O `docker-compose` usa `prometheus.docker.standalone.yml`, que faz scrape em `host.docker.internal:8000` e `host.docker.internal:9092`. Se o Alertmanager impedir o início do Prometheus, use a versão standalone (alerting comentado) ou suba o Alertmanager antes.

Exemplos de queries PromQL

Objetivo	Query
Taxa de requisições (últimos 5 min)	`rate(asis_requests_total[5m])`
Latência P95 por endpoint	`histogram_quantile(0.95, sum(rate(asis_request_duration_seconds_bucket[5m])) by (le, endpoint))`
Latência P90 (SLA DN1)	`histogram_quantile(0.9, sum(rate(asis_request_duration_seconds_bucket[5m])) by (le))`
Taxa de erro (DN3)	`asis_health_error_rate_percent`
Estado do circuit breaker	`asis_circuit_breaker_state`
Tamanho do ProcessStore	`asis_process_store_size`
Tamanho do AuditLog	`asis_audit_log_size`
Status do target da API	`up{job="asis_api"}`
Total de requisições	`asis_requests_total`

Regras de alerta

As regras em `rules.yml` incluem: **HighLatency** (P90 > 650ms), **SLAViolation** (P90 > 500ms), **HighErrorRate** (> 30%), **CircuitBreakerOpen**, **RateLimitLow**, **AuditChainAlert**. O `alertmanager.yml` roteia por severidade (critical, warning).

Alertmanager

O **Alertmanager** recebe alertas disparados pelo Prometheus, agrupa, deduplica e encaminha notificações para os receivers configurados. Interface web: http://localhost:9093.

Roteamento:

Severidade	Receiver	Alertas encaminhados
`critical`	`critical`	SLAViolation (DN1), CircuitBreakerOpen (DN3), AuditChainAlert (DN4)
`warning`	`warning`	HighLatency (DN1), HighErrorRate (DN3), RateLimitLow (DN3)
Demais	`default`	Agrupamento sem notificação externa

Configuração (`alertmanager.yml`):

- `group_by`: `['alertname', 'severity']` — agrupa alertas pelo nome e severidade
- `group_wait`: 30s — espera antes de enviar o primeiro grupo
- `group_interval`: 5m — intervalo entre envios de grupos
- `repeat_interval`: 5m — intervalo para reenviar alertas ainda ativos

Receivers: Os receivers `default`, `critical` e `warning` estão definidos sem canais de notificação (webhook, Slack, email). Para ativar envio, adicione em `alertmanager.yml` exemplos como `webhook_configs`, `slack_configs` ou `email_configs` em cada receiver.

6.2 Grafana e Dashboards Executivos

O Grafana é provisionado automaticamente via Docker Compose com datasource Prometheus e 4 dashboards pré-carregados. A camada de visualização é complementada por um frontend Next.js que organiza leituras orientadas à decisão.

Dashboards Provisionados

Dashboard	UID	JSON	Foco
Dashboard da Qualidade	`asis-quality-dashboard`	`grafana/dashboards/asis-quality-dashboard.json`	25 painéis técnicos: DN1-DN4, RF1/RF2, Massa de Testes, Alertas
Dashboard de Negócio	`asis-business-dashboard`	`grafana/dashboards/asis-business-dashboard.json`	Leitura executiva: SLA, throughput, sucesso, impacto operacional
Dashboard de Saúde	`asis-system-health-dashboard`	`grafana/dashboards/asis-system-health-dashboard.json`	Saúde operacional: error rate, circuit breaker, rate limit, alertas
Dashboard de Testes	`asis-testing-gap-dashboard`	`grafana/dashboards/asis-testing-gap-dashboard.json`	Cobertura e gaps de teste

Frontend Executivo (Next.js)

A aplicação em `dashboard/frontend/` serve como camada de apresentação para três contextos complementares:

Rota	Público-alvo	Função
`/business`	Liderança, gestores	Traduz sinais técnicos em leitura de impacto operacional
`/system-health`	Engenharia, SRE, operação	Triagem rápida de incidentes e degradações
`/ai-search`	Qualquer usuário	Busca inteligente de painéis por linguagem natural (OpenAI + fallback local)

Endpoints Backend de Dashboard

Endpoint	Descrição
`GET /api/v1/dashboard/business`	Dados agregados de negócio (summary, trends, degradation)
`GET /api/v1/dashboard/system-health`	Dados de saúde do sistema (health_score, error_rate, circuit_breaker)
`GET /api/v1/dashboard/testing-gap`	Dados de cobertura de testes
`GET /api/v1/dashboard/metadata`	Metadados e filtros disponíveis

Testes Blue Team dos Dashboards

A suíte em `src/tests/blue_team/dashboards_blue.py` valida o contrato dos endpoints de dashboard:

- `test_dashboard_business_retorna_contrato_basico`: verifica status 200, campos `summary` e `trends`
- `test_dashboard_system_health_retorna_contrato_basico`: verifica status 200, campos `summary` e `breakdown`
- `test_dashboard_testing_gap_retorna_contrato_basico`: verifica status 200, campos `summary` e `coverage`
- `test_dashboard_metadata_retorna_filtros`: verifica status 200, campos `dashboards` e `filters`
- `test_dashboard_rejeita_interval_invalido`: verifica rejeição de parâmetros inválidos

```
pytest src/tests/blue_team/dashboards_blue.py -v
```

Documentação Complementar

Para detalhes técnicos sobre a stack de observabilidade, painéis PromQL, Truth Definitions e configuração:

- **dashboard.md**: documentação completa do Dashboard da Qualidade (métricas, painéis, variáveis template, regras de alerta)
- **execucao-dashboard.md**: guia unificado de execução da stack completa (API, Docker, Grafana, frontend)
- **plano_integracao_prometheus.md**: plano técnico de integração Prometheus
- **catalogo_servicos_monitorados.md**: catálogo de endpoints e mapeamento para DN/RF/RNF

---

7. Consolidação Final da API e do Controle de Qualidade

7.1 Ajustes finais de implementação da API

Nesta etapa final, a API foi consolidada com foco em robustez funcional, segurança de acesso, rastreabilidade e operação observável.

Ajustes aplicados na camada de domínio da API

- **Validação semântica de destino fiscal (RF2):** o campo `destino` passou a ser validado contra `MODULOS_FISCAIS_VALIDOS`, com normalização case-insensitive e erro HTTP 400 para módulos não suportados.
- **Autenticidade de credenciais (RNF2):** a autenticação por `account-key` e `app-key`

foi reforçada com registro explícito de pares válidos (`VALID_CREDENTIAL_PAIRS`), com retorno 401 para ausência de credenciais e 403 para credenciais inválidas.
- **Consulta de status assíncrono (RF1):** a API mantém estado de processamento em `ProcessStore` e expõe consulta via `GET /api/v1/processo/{tracking_id}` para rastreamento do ciclo do lote.

Ajustes aplicados na camada de resiliência (DN3)

- **Rate limiting por conta** com janela deslizante (`RateLimiter`) e visibilidade de saldo remanescente por chave hasheada.
- **Circuit breaker com estados `CLOSED` / `OPEN` / `HALF_OPEN`** (`CircuitBreaker`) para evitar falhas em cascata.
- **Health check operacional** com métricas de erro, uptime e volume processado (`HealthChecker`).
- **Idempotência** com store dedicado (`IdempotencyStore`) para evitar reprocessamento acidental de requisições.
- **Degradação graciosa e alertas de disponibilidade**, com resposta de fallback e sinalização explícita quando o serviço opera sob pressão.

Ajustes aplicados na camada de não-repúdio (DN4)

- **Assinatura HMAC-SHA256** por transação, vinculando payload e timestamp.
- **Recibo digital completo** com `signature`, `timestamp`, `actor`, `tracking_id` e `payload_hash`.
- **Log de auditoria encadeado (hash chain)** por meio de `AuditLog`, com verificação de integridade ponta a ponta.
- **Endpoint de disputa de não-repúdio** (`POST /api/v1/disputa/nr`) integrado ao histórico transacional para análise de sazonalidade.

Ajustes aplicados na camada de identidade (INT3)

- **Autenticação híbrida consolidada:** convivência de API Keys legadas com Clerk JWT no endpoint `POST /api/v1/auditar/clerk`.
- **Validação JWT com JWKS e cache com TTL**, reduzindo latência e dependência de chamadas externas a cada requisição.
- **Pseudonimização de identidade** (hash de ator) para manter rastreabilidade com aderência à LGPD.

Ajustes aplicados na camada de observabilidade

- **Instrumentação Prometheus no núcleo da API** (`src/core/metrics.py`) com métricas de latência, integridade, disponibilidade, não-repúdio e capacidade.
- **Endpoint `GET /metrics`** consolidado para integração com Prometheus e dashboards de operação/qualidade.
- **Serviços de dashboard backend** (`/api/v1/dashboard/business`, `/system-health`, `/testing-gap`, `/metadata`) estabilizados para consumo do Grafana e do frontend Next.js.

7.2 Análise do repositório: programas de controle de qualidade

O repositório ficou estruturado com um conjunto de programas de controle de qualidade em múltiplas camadas (teste, análise estática, carga, observabilidade e governança de versão).

Visão consolidada dos programas

Programa	Objetivo	Implementação no repositório	
Testes funcionais e não funcionais (Blue Team)	Validar contrato, regras de negócio e critérios de aceite por DN/RF/RNF	`src/tests/blue_team/*.py`	`pyt` `src`
Testes adversariais (Red Team)	Exercitar abuso, bypass e limites operacionais	`src/tests/red_team/*.py`	`pyt` `src`
Análise estática e quality gate (INT1)	Medir qualidade de código e bloquear regressão estrutural	`.gitlab-ci.yml` + `sonar-project.properties` + SonarCloud	`sta` `sca`

Cobertura e relatório de testes	Garantir testabilidade e rastreabilidade de execução	`pytest-cov`, `pytest-html`, `coverage.xml`, `pytest-report.xml`	`pyt`
			`-co`
			`rep`
			`--`
			`rep`

Carga e estresse (INT2 / Locust)	Medir comportamento sob concorrência real e validar SLA	`src/locust_tests.py` + documentação de execução	`loc` `htt`
Métricas de massa de testes	Expor qualidade da suíte para monitoramento contínuo	`src/scripts/collect_pytest_metrics.py` e `src/scripts/pytest_metrics_exporter.py`	`exp` `:9`
Observabilidade operacional	Correlacionar saúde da API com DNs e alertas	`prometheus.docker.standalone.yml`, `rules.yml`, `docker-compose.yml`, dashboards Grafana	`sta` `Gra`
Governança de versão (INT4)	Verificar sincronismo entre homologação e produção	`src/scripts/verifica_versao_gitlab.py` + testes Blue/Red	`cor` `dev`

Síntese do estado final dos controles de qualidade

- O repositório adotou **controle de qualidade orientado a evidência**, com rastreio explícito entre requisitos (DN/RF/RNF), código e testes.
- A qualidade foi tratada como **processo contínuo**, combinando validação pré-merge (CI), monitoramento em runtime (Prometheus/Grafana) e testes adversariais.
- A cobertura inclui tanto **conformidade funcional** quanto **resiliência operacional e segurança**, reduzindo risco de regressões silenciosas.
- A arquitetura documental e técnica está pronta para evolução incremental, com trilha clara para endurecimento adicional de produção (políticas de retenção, limites de store em memória, e expansão de controles de autenticação/anti-replay).